

Anonymous Credentials from ECDSA

Matteo Frigo  and abhi shelat 

Google, Cambridge, MA, USA

Abstract. Anonymous digital credentials allow a user to prove possession of an attribute that has been asserted by an identity issuer without the user revealing any extra information about themselves. For example, a user who has received a digital passport credential can prove their “age is > 18 ” without revealing any other attributes such as their name or date of birth.

Despite their clear application to privacy-preserving authentication, anonymous credential schemes have been difficult to deploy at scale. Part of the difficulty arises because schemes in the literature, such as BBS+ [CDL16], use new cryptographic primitives that require system-wide changes to existing issuer infrastructure. In addition, issuers often require digital identity credentials to be *device-bound* by incorporating the device’s secure element into the presentation flow. As a result, schemes like BBS+ require updates to the hardware on every user’s device.

We propose new ZK techniques which enable the construction of an anonymous credential scheme for the legacy Elliptic Curve Digital Signature Algorithm (ECDSA) signature scheme. By adding efficient ZK arguments for statements about SHA256 and document parsing for ISO-standardized identity formats, we construct the first ZK proof of possession of a credential that can be deployed *without* changing any issuer processes, *without* changes to mobile devices, and *without* requiring non-standard cryptographic assumptions. Furthermore, our proof system itself only relies on SHA-256 as its complexity assumption.

Producing ZK proofs about ECDSA signatures has been a bottleneck for other ZK proof systems because standardized curves such as P256 use finite fields which do not support efficient number theoretic transforms. We overcome this bottleneck by designing a ZK proof system around sumcheck and the Ligero argument system, by designing efficient methods for Reed-Solomon encoding over the required fields, and by designing specialized circuits for ECDSA.

Our proofs for ECDSA can be generated in as little as ≈ 20 ms. When incorporated into a fully standardized identity protocol such as the ISO MDOC standard, our system can generate a zero-knowledge proof for the MDOC presentation flow in a few hundred ms on mobile devices. These advantages make our scheme a promising candidate for privacy-preserving digital identity applications.

1 Introduction

Chaum [Cha85] introduced the idea of an *anonymous credential system*, or *pseudo-nym system* as a method to provide individuals full control over the dissemination of personal information. Such a system consists of users, issuers, and relying parties. Issuers give a set of digital credentials to a user; users can prove to a relying party that they possess credentials satisfying a property without revealing anything more than the fact that they hold credentials that satisfy the property. Colluding relying parties are unable to link a specific user to a pair of interactions they participate in. Even a colluding issuer and relying party should not be able to link a specific user to a session.

E-mail: matteof@google.com (Matteo Frigo), shelat@google.com (abhi shelat)



Early constructions of such systems based on specific signature schemes imposed unwieldy restrictions on usage. Lysyanskaya, Rivest, Sahai and Wolf [LRSW99] devised a scheme based on one-way functions and generic zero-knowledge protocols. They proposed a more efficient construction based on a non-standard discrete-logarithm assumption, but it only supported unlinkability under restrictions on credential issuance and usage. Brands [Bra99] introduces a selective disclosure system based on certificates, but it does not support unlinkable reuse of credentials.

Camenisch and Lysyanskaya [CL01] introduced a bespoke credential system that satisfied all of the multi-use and unlinkability properties desirable in a credential system, but their scheme was both computationally expensive and relied on a non-standard cryptographic assumption. They observed that a key problem for an anonymous credential scheme is to design a signature scheme for which there is also an efficient proof of knowledge of a signature. Later, Camenisch, Drijvers, and Lehmann presented the BBS+ credential scheme [CDL16] as an efficient method to prove knowledge of a signature and thus construct an anonymous credential system.

Kosba et al. [KZM⁺15] propose the use of general purpose zkSNARKS to build anonymous credentials based on *standardized* signature schemes. In one of their schemes, a credential consists of a 1KB message that is signed using RSA-PSS with a 2048-bit key and SHA-256 as the hash function. A user can prove one inequality between a value in the message and an external one (i.e., the basic age-over-X application). Their prover runs in 54–108s depending on parameters. Later, Delignat-Lavaud, Fournet, Kohlweiss, and Parno [DLFKP16] constructed a zkSNARK system for proving possession of an X509 certificate for RSA signatures, which requires parsing the ASN.1 format in the ZK circuit. Their system took 31s to produce just the RSA signature proof, and took between 61-160s for the entire anonymous credential presentation. In addition to the impractical time to present a credential, both of these proof systems also require a costly *trusted parameter setup*, and they also require using pairing-friendly curves like the BBS+ scheme.

Rosenberg et al. [RWGM23] build anonymous credentials using general ZK techniques applied to *non-standardized* signature and hash schemes. Their system exploits a blockchain to establish *common knowledge*: an issuer must augment a public data structure when they issue a credential to a user. The user refers to this public datastructure when they present a credential. As a result, their system reduces the credential presentation problem to a ZK proof of set membership. In addition, they use the (non-standard) Poseidon hash function to make more efficient ZK proofs concerning set membership. Altogether, the addition of the blockchain and the use of Poseidon allows their proof system to run between 460–542ms. An alternative formulation using (non-standard) Schnorr signatures over a (non-standard) pairing-friendly curve runs faster in 130ms. Unfortunately, their prover requires up to 800mb of memory to produce a proof, and as before, a *trusted parameter* that is several 100s of megabytes *in addition* to relying on pairing-friendly curves.

Impediments While there have been attempts to deploy credential systems for digital identity applications, such efforts encounter three major roadblocks.

First, many identity issuers have deployed hardware security modules that only support *standardized* RSA or ECDSA signatures. Such organizations may be unwilling or incapable of updating their infrastructure to support credential systems that rely on newer cryptographic assumptions such as pairing-friendly curves or lattices.

A second problem is that credentials are issued by multiple organizations (e.g., state DMVs), and relying parties need to support all of them. The task of producing a *single* common trusted parameter via a secure multi-party computation ceremony involving *all* issuers seems highly impractical. This suggests that even the proposed anonymous credential schemes discussed above that are based on *standardized cryptography* are unsuitable because their proof systems require bilinear pairings and large trusted setup parameters.

Another critical impediment is the potential for *scaled abuse* of anonymous credentials. If credentials can be copied from one malicious user to another, then the system may not be able to provide any guaranteed soundness. Because an anonymous credential does not leak any information other than the shared attributes during a presentation flow, it is more difficult to discover and/or revoke copied credentials just by observing the presentation flows. This problem arises even in the setting of certificate systems that only offer selective disclosure properties.

To address this *copied credential* problem, issuers of digital identity credentials require users to hold their credential using trusted hardware. Modern mobile devices often contain a *secure element* which is a hardware module designed to be tamper-resistant, side-channel resistant, and thus secure against most key extraction attacks. The identity protocols in the ISO MDOC standard, for example, specify issuance and presentation flows that require the secure element on the receiving device to store a portion of the credential. Most deployed secure elements only support legacy standardized signature schemes such as RSA and ECDSA, and in particular, they do not support performing operations on pairing-friendly curves, lattices, or even well-deployed signature schemes such as EdDSA. It requires a decade to upgrade these elements in the field, which makes the adoption of schemes such as BBS+ impractical.

1.1 Anonymous Credentials from ECDSA

To address all of the noted shortcomings, namely (a) efficiency, (b) requirement to use standardized signature schemes, and (c) lack of trusted setups, we present new methods to construct all of the components necessary to build a performant anonymous credential that relies only on ECDSA signatures.

The main contribution in this paper is the construction of an efficient proof of knowledge of an ECDSA signature; we also develop the other tools necessary for an issuer to issue a set of credentials or attributes to a user. We then combine these into a protocol that allows a user to demonstrate possession of an arbitrary subset of those credentials to a relying party. Technically, this step yields a proof of possession of a signature, which can be bootstrapped to an anonymous credential using standard techniques for pseudo-nym creation from a PRF. This paper will focus on constructing and benchmarking these key primitives.

In contrast to prior work on ZK for ECDSA (see §5.3 for comparisons), our proposed proof of knowledge of an ECDSA signature runs in almost the same time as a BBS+ proof, and it only relies on the security of a collision-resistant hash function such as the SHA256 function. The key to our approach is to combine two recent ZK proof techniques: an optimized verifiable computation protocol that is based on sumcheck, and the Ligero ZK proof system [AHIV23] that is based on the IOP (or more precisely, IPCP) model. The use of sumcheck in recent ZK constructions [WTas⁺18] shows promise: it avoids having to compute an super-linear time Number Theoretic Transform (NTT) on the full witness

for a given circuit. Instead, it is possible to run a sumcheck-based verification protocol in roughly *linear* time in the size of the theorem-verifying circuit. Moreover, verifiable computation protocols that are based on sumcheck are information-theoretically secure as they require no computational assumptions. Similarly, the Ligero proof system in the IPCP model requires the instantiation of a secure *proof oracle*, which can also be achieved using Merkle trees that only rely on the random oracle. In practice, we instantiate our protocols in a non-interactive manner, and thus rely on the random oracle model for the soundness and zero-knowledge properties.

Key Results One obstacle to proving knowledge of an ECDSA signature is common to elliptic curves that have been standardized. Standardized curves for ECDSA, such as the NIST P256 curve, specify finite fields that do not have the roots of unity needed for efficient NTT algorithms. Consequently, prior work *emulates* the arithmetic of the P256 field in *another* finite field that is optimized for ZK proofs. Even using very clever arithmetic tricks, this overhead amounts to 40–100x increase in arithmetic operations.

We overcome this bottleneck using several techniques. First, we design an NTT in a quadratic field extension which allows us to perform NTTs (in fields that do not have proper roots of unity) at only $\approx 1.5x$ the cost of doing so in an NTT-friendly prime of the same size.

We also develop a novel Reed-Solomon encoding approach which can exploit the subfield structure of the field, yielding a constant-factor improvement in proof size over the standard NTT-based methods. Instead of defining the Reed-Solomon code as a polynomial interpolation over the roots of unity, we define the code as an interpolation over an arithmetic sequence of evaluation points in the base field. We present a method to perform the interpolation over these points using a linear convolution. Thus, when our witness values are over a small subfield or base field, even though the NTT itself may be carried out over a larger extension field, the resulting codeword remains in the subfield, yielding a smaller proof.

Some applications require zero-knowledge arguments about the pre-image of certain *naturally* boolean functions such as SHA256. Following the work of Diamond and Posen [DP25, DP24], we confirm the speedup possible with using the extension field $\text{GF}(2^k)$ to perform the sumcheck. In this case, the standard NTT algorithms no longer apply, and we instead use the family of additive FFTs [LCH14]. Specifically, we develop a new *bidirectional truncated* additive FFT to solve the problem of polynomial interpolation over cosets. As our benchmarks show, this operation provides almost an order of magnitude speedup in basic algebraic operations.

Our next contribution stems from the specific circuits we developed for verifying ECDSA signatures, verifying the pre-image of SHA-256, and for parsing nested document formats such as CBOR that are specified in current identity credential standards such as ISO MDOC. These components are necessary in order to satisfy our goal of constructing proof of possession for *legacy* credential formats.

We also present a method to improve performance when a ZK credential verification task involves both ECDSA signatures and SHA-256 hashes. Prior proof systems verify a single arithmetic circuit defined over one finite field $\mathbb{F}$. However, as our benchmarks show, it is substantially faster to verify properties about SHA-256 preimages using a binary field extension instead of a prime field. A natural approach is to verify parts of a statement under a circuit over $\mathbb{F}_p$, while verifying other constraints under a more convenient field $\text{GF}(2^k)$. A problem arises when a common witness value must be used in both parts of this circuit. A malicious prover may include inconsistent copies of this witness value in the two circuits. To address this problem, we design a *witness consistency protocol* in which witnesses that are committed in two different ZK proofs over different fields can be proven to be consistent with high probability. Our approach is to include the evaluation

of a random MAC on the witness—a computation that is small and easy to simulate over any field in both $\mathbb{F}$ and $\text{GF}(2^k)$. We measure the overhead of this consistency check and show that after the SHA-256 pre-image verification exceeds a few blocks, our consistency method becomes more efficient.

Special notes about this application Some recent *succinct* ZK NARK systems produce proofs on the order of hundreds of bytes by exploiting a very large trusted parameter setup. In an anonymous credential system that involves many issuers, relying parties, and users, such a globally trusted setup ceremony is impractical to coordinate. This paper focuses on ZK solutions that *do not* require any trusted setup.

Additionally, while the system we develop is succinct in proof size, many recent ZK systems are also succinct in verifier-time. In our anonymous credential application, however, it is more important to conserve *prover time*, because the prover is often the user’s mobile device.

Contemporaneous work Woo et al. [WOS⁺25] present an independent construction of a ZK proof of possession for legacy signatures such as ECDSA. Their construction takes a different strategy and results in a proof for ECDSA that is several times slower and also requires the public key of the signer to be a component of the theorem statement (thereby making it unsuitable, for example, to be applied in the MDOC protocol). This paper includes new techniques for sumcheck, RS encoding, and two-field theorems, as well as different techniques for arithmetizing ECDSA and SHA-256. See §5.3 for more discussion.

Open source availability The system described in this paper is available as open source at <https://github.com/google/longfellow-zk>.

2 Zero-Knowledge Argument System

We construct a zero-knowledge argument (ZKARG) system by composing the Ligerio proof system [AHIV23] with a public-coin verifiable computation (VC) protocol that is based on the sumcheck protocol. A VC protocol allows a prover to convince a verifier that $C(x) = 0$ in time and space that is smaller than $|C|$. Such protocols have been implicit in the literature since the introduction of sumcheck [LFKN92]. See Thaler [Tha22] for standard cryptographic definitions (omitted here for space).

Our approach to constructing a ZK argument follows the one of Hyrax [WTas⁺18], which is an instantiation of the idea from Ben-Or et al. [BOGG⁺88]. Specifically, on input a circuit C , public input x , and witness w , the prover first sends a commitment com to w and commitment parameters. The prover and verifier then interactively produce a *committed* transcript T for a VC protocol on input (C, x) ; specifically, when the VC protocol instructs the prover to send its i^{th} message m_i to the verifier, it instead sends a commitment t'_i to that message; the verifier always responds to the prover’s i^{th} commitment with a random challenge as per the usual public-coin VC protocol and defers performing all of the prescribed verification steps. Finally, the prover and verifier engage in a zero-knowledge proof for the theorem “ $(t'_1, \dots, t'_\ell)$ is a commitment to transcript t and t is an accepting VC transcript for (C, x) .” In particular, the ZK proof for this theorem implicitly ensures that all of the verifier’s VC steps hold with respect to the original transcript t .

The resulting composition enjoys efficiencies. Because VC produces a succinct proof with respect to C , the Ligerio proof in our system is *asymptotically smaller* than a standalone Ligerio proof for $C(x) = 0$. As we show below, this significantly improves the performance of the overall system because the most costly component of Ligerio, the NTT, is applied to

a small transcript for $C(x) = 0$ instead of a full tableau for the computation of $C(x)$. Note that the VC protocol is not a zero-knowledge protocol, but Liger is, and in this case, the composition yields a protocol that satisfies the zero-knowledge property.

In the rest of this section, we explain each component in more detail, and provide more formal descriptions of the protocol and its performance.

2.1 Verify Interactive Protocol

We describe a verifiable computation (VC) protocol derived from a line of work based on the sumcheck protocol [GKR08, CMT12, Tha13, WJB⁺17, WTas⁺18].

Sumcheck The sumcheck interactive protocol [LFKN92] allows a Prover to efficiently convince a Verifier that a claimed value equals the sum of a polynomial evaluated at every point on the boolean hypercube. Here we quote a performance theorem taken from Thaler [Tha22] characterizing sumcheck.

Theorem 1 ([Tha22, Proposition 4.1]). *Let g be a v -variate polynomial of degree at most d in each variable, defined over a finite field $\mathbb{F}$. For any specified $H \in \mathbb{F}$, let $\mathcal{L}_H$ be the language of polynomials g (given as an oracle) such that*

$$H = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_v \in \{0,1\}} g(b_1, \dots, b_v). \quad (1)$$

The sum-check protocol is an interactive proof system for $\mathcal{L}_H$ with completeness error $\delta_c = 0$ and soundness error $\delta_s \leq vd / |\mathbb{F}|$.

Moreover, if $\deg(g) = O(1)$, then the protocol requires v rounds of communication, the total communication cost is $O(v)$ field elements, the verifier time is $O(v) + T(g)$, and the prover's time is $O(M(g))$ where T represents the cost of 1 evaluation of g and M represents the cost of evaluating g on the hypercube.

As a notational convenience, we often use capital letters to represent a vector of variables over $\{0, 1\}$. For example, letting $B = (b_1, \dots, b_v)$, we write:

$$H = \sum_{B \in \{0,1\}^v} g(B)$$

or even $H = \sum_B g(B)$ as shorthand for Equation (1).

Applying sumcheck for circuit verification Thaler [Tha22] provides background on sumcheck and an overview of the application of sumcheck to the task of verifiable computing. In particular, as first shown in [GKR08], sumcheck can be applied to the task of verifying that $C(x) = 0$ where C a *layered circuit* starting with layer 0 as the output layer, and d as the input layer.

We present an optimized variant in this family of protocols that builds on observations by Cormode et al. [CMT12], Thaler [Tha13], and other follow-ups [WJB⁺17, WTas⁺18]. Our improvements consist of a simplification of the wiring predicate, a different approach to the layer-to-layer reduction, and different methods to compute the evaluation of the sparse wiring predicate.

In our formulation, layer j of the circuit computes an array $V_j[0, \dots, w_j - 1]$ of output wires given an array $V_{j+1}[0, \dots, w_{j+1} - 1]$ of input wires, which are the outputs of layer $(j + 1)$. Let $\omega_j = \lceil \log_2(w_j) \rceil$. Instead of the separate addition and multiplication gates of [GKR08], we support only one (degree-2) “gate” which computes the generic quadratic form:

$$V_j[i] = \sum_{\ell, r \in \{0,1\}^{\omega_{j+1}}} Q_j[i, \ell, r] \cdot V_{j+1}[\ell] \cdot V_{j+1}[r],$$

We call Q the *quad* and $Q_j[i, \ell, r]$ the *quad term*.

By convention, the input wire $V_j[0] = 1$ for all layers, and thus the quad representation handles the *add* and *multiplication* gates in a uniform manner. For example, this quad form approach reduces the depth of the standard elliptic curve addition formula from 5 to 3. By convention, an input satisfies the circuit if all outputs $V_0[i]$ of layer 0 are 0.

The *arrays* defined above naturally specify multi-variate—and specifically multi-linear—polynomials. We use tilde-notation to denote the unique multi-linear extension for an array. For example, $\tilde{V}_j(i)$ represents the unique multi-linear extension that interpolates the array $V_j[i]$.

At the beginning of the protocol, the verifier picks a random point $r_0 \in \mathbb{F}^{\omega_0}$. Because outputs are expected to be 0, the verifier expects $\tilde{V}_0(r_0) = 0$. Our process begins with *two* claims $(c_{0,0}, c_{0,1}) = (0, 0)$ on the values $\tilde{V}_0(r_0)$. In this first step, both claims are on values on the same evaluation point, but in later rounds, these claims will be on different points.

Protocol 2.2. Verify_{P,V}(C, x) for statement $C(x) = 0$

The depth- d circuit C is defined over field $\mathbb{F}$. Prover and Verifier hold input (C, x) . Let $(Q_0, \dots, Q_{d-1})$ be the quad form for each layer of C , and define w_j to be the number of wires in layer j and $\omega_j = \lceil \log_2(w_j) \rceil$. Let $V_j[i]$ represent the value of the i^{th} output wire in layer j . Verifier rejects if any check fails.

1. Verifier samples $r \leftarrow \mathbb{F}^{\omega_0}$ and sets $r_{0,b} \leftarrow r, c_{0,b} \leftarrow 0$ for $b \in \{0, 1\}$.
2. For $j = 0, 1, \dots, d-1$: // Prover asserts $c_{j,b} = \tilde{V}_j(r_{j,b}), b \in \{0, 1\}$.
 - (a) Verifier sends $\alpha_j \leftarrow \mathbb{F}$ and sets $Y = \alpha_j \cdot c_{j,0} + (1 - \alpha_j)c_{j,1}$.
 - (b) Define the $2\omega_{j+1}$ -variate polynomial

$$g_j(L, R) = [\alpha_j \cdot \tilde{Q}_j(r_{j,0}, L, R) + (1 - \alpha_j)\tilde{Q}_j(r_{j,1}, L, R)] \cdot \tilde{V}_{j+1}(L) \cdot \tilde{V}_{j+1}(R)$$

in $L = L_0, \dots, L_{\omega_{j+1}-1}$ and $R = R_0, \dots, R_{\omega_{j+1}-1}$.

- (c) Prover and Verifier apply sumcheck to the claim:

$$Y = \sum_{L, R \in \{0, 1\}^{\omega_{j+1}}} g_j(L, R)$$

The sumcheck protocol produces two random points ℓ and r , a value y , and a claim that $y \stackrel{?}{=} g_j(\ell, r)$.

- (d) Prover sends claims $c_{j+1,0} = \tilde{V}_{j+1}(\ell)$ and $c_{j+1,1} = \tilde{V}_{j+1}(r)$.
- (e) Verifier checks:

$$y \stackrel{?}{=} [\alpha_j \cdot \tilde{Q}_j(r_{j,0}, \ell, r) + (1 - \alpha_j)\tilde{Q}_j(r_{j,1}, \ell, r)] \cdot c_{j+1,0} \cdot c_{j+1,1}$$

and continues with points $(r_{j+1,0}, r_{j+1,1}) = (\ell, r)$ and claims $c_{j+1,b}$.

3. Verifier verifies the claims $c_{d,b} \stackrel{?}{=} \tilde{V}_d(r_{d,b})$ by direct evaluation of $\tilde{V}_d(r_{d,b})$.

The next theorem is adapted from a similar one in Thaler [Tha22].

Theorem 3 ([Tha22]). Let C be a depth- d layered circuit over $\mathbb{F}$ whose j^{th} layer has width w_j , let $\omega_j = \lceil \log(w_j) \rceil$ and let $Z = 2 \sum_j^d \omega_j$.

$\text{Verify}_{P,V}(C, x)$ is a Z -round interactive protocol for the theorem $C(x) = 0$ that has completeness error 0, soundness error $O(Z/|\mathbb{F}|)$ and total communication $\Theta(Z)$ field elements.

Moreover, the verifier computation consists of sampling $\Theta(Z)$ elements from $\mathbb{F}$, performing $\Theta(Z)$ operations in $\mathbb{F}$, and performing one dot product between the input x and a vector with tensor form derived from the randomly sampled challenges. The Prover computation consists of $O(|C|)$ field operations.

The notation $\text{sz}(C, x)$ represents this protocol's transcript length on instance (C, x) .

Remarks Achieving the stated runtime requires careful, but well-known optimizations of sumcheck for the Prover and Verifier and an efficient method to evaluate the wiring predicate $\tilde{Q}$ at each layer. Step 2e can be moved outside of the loop. Moreover, all of the verifier's checks in Sumcheck in step 2c can also be moved outside of the loop. In this formulation, the Verifier can be split into the interactive part that includes step 1, 2(a), and the random sampling steps in 2(c), and a second part that performs all of the checks in steps 2c, 2e, and 3. We denote this second part as $\text{check}_V(C, x)$.

SIMD optimization It is well known that when C validates multiple copies of the same smaller circuit, it is possible to achieve verifier succinctness or speedup in time [CMT12, Tha13, WJB⁺17]. These extensions can be added to our formulation, but are omitted because our system does not require verifier succinctness.

2.2 Liger Zero-Knowledge System

The Liger [AHIV23] protocol is a zero-knowledge interactive PCP that represents an optimized instantiation of the "MPC-in-the-head" technique. The proof size is roughly square-root in the size of the circuit, and the proof system only assumes the existence of a cryptographic hash function. Note that an IPCP is a special case of an IOP in which the prover sends a proof oracle only in the first round, and subsequent rounds only require a standard message response to the verifier's challenge instead of an oracle¹.

Whereas the previous section considered a circuit C with input x , we now further split the input into a pair (x, w) where x represents the *public input* that is known to both Prover and Verifier, and w is the *private witness* that is known only to the Prover. See Liger [AHIV23] for standard definitions of ZK Proof of Knowledge.

We now identify the components of the Liger protocol. The $\text{LigerCommit}(x, w)$ method allows a Prover to produce a short commitment com to inputs (x, w) . Next, prover and verifier respectively run LigerProver on inputs (C, x, w, com) and LigerVerify on inputs (C, x, com) , and finally, the verifier method outputs accept or reject. Ames et al. [AHIV23] characterize the performance of Liger:

Theorem 4 ([AHIV23, 5.4]). *Fix parameters n, m, ℓ, k, t, e such that $e < (n - k)/2$ and $n > 2k + e$. Let $C: \mathbb{F}^{n_i} \rightarrow \mathbb{F}$ be an arithmetic circuit of size s , where $|\mathbb{F}| \geq \ell + n$, $m \cdot \ell > n_i + s$ and $k > \ell + t$. Then the non-interactive protocol (P', V') compiled from protocol $\text{ZKIPCP}(C, \mathbb{F})$ using the BCS16-transformation is a non-interactive random-oracle proof with:*

- Adaptive soundness error $\epsilon_{FS}(C, T, \lambda) = T \cdot \epsilon(C) + 3(T^2 + 1)/2^\lambda$ and adaptive knowledge error $\epsilon_{FS-K}(C, T, \lambda) = T \cdot \epsilon(C) + 3(T^2 + 1)/2^\lambda$ against T -query provers;
- Adaptive soundness error $\epsilon_{QFS}(C, T, \lambda) = \Theta(T \cdot \epsilon_{FS}(C))$ and adaptive knowledge error $\epsilon_{QFS-K}(C, T, \lambda) = \Theta(T \cdot \epsilon_{FS}(C))$ against $T - O(q(C) \log(\text{len}(C)))$ -query quantum provers;

¹The treatment of subsequent Prover communication as messages instead of oracles consisting of a single point enables a simpler and more efficient compilation from IOP to protocol.

- *Statistical zero-knowledge* $z'(C, \lambda) = 4\text{len}(C)/2^{\lambda/4}$

where

$$\epsilon(C) = \epsilon_K(C) = \max \left(\frac{(n+2)}{|\mathbb{F}|}, (1 - e/n)^t \right),$$

$$\text{len}(C) = n + (k + \ell - 1) + (2 \cdot k - 1)$$

and V' makes t queries of symbols of size $\mathbb{F}^{4m+5}$.

Improvement In Appendix C of an updated version of [AHIV23], the authors conclude that the overall soundness error can be bounded by

$$\left[(1 - e/n)^t + \left(\frac{k + \ell}{n} \right)^t + \left(\frac{2k}{n} \right)^t + \frac{n+3}{|\mathbb{F}|} \right]$$

We use this formula when setting security parameters.

2.3 ZK Protocol

Our first contribution is to note the efficiency of using the Ligero zero-knowledge protocol to *verify* the check_V verifier. This ultimately yields a zero-knowledge protocol for a circuit $C(x, w) = 0$ that is substantially more efficient than using Ligero (or as we show, any other previously known setup-free ZK system) for the same circuit.

Protocol 2.5. ZK protocol (P, V) for circuit $C(x, w) = 0$

The circuit C is defined over field $\mathbb{F}$. Prover holds input $(C, (x, w))$ and Verifier holds input (C, x) .

1. P, V compute the size $\ell \leftarrow \text{sz}(C)$ of the Verify transcript.
2. P samples $\text{pad} \leftarrow \mathbb{F}^\ell$ and sets $w' \leftarrow (w, \text{pad})$ as the witness.
3. P computes $\text{com} \leftarrow \text{LigeroCommit}(w')$ and sends com to V .
4. P, V run $\text{Verify}_{P,V}(C, (x, w))$ with these changes:
 - Whenever Verify_P instructs the prover to send the i^{th} message $m_i \in \mathbb{F}$ to V , P sends $t'_i \leftarrow m_i + \text{pad}_i$.
 - When Verify_V instructs the verifier to send the i^{th} random challenge r_i , V returns r_i .

At the end, P, V hold an encrypted sumcheck transcript $T' = ((t'_i, r_i))_{i \in [0, \ell]}$ for instance $(C, (x, w))$.

5. Define the circuit $C'(x, w', T')$ to first *decrypt* the transcript $T = ((t_i = t'_i - \text{pad}_i, r_i))_{i \in [0, \ell]}$, and then run $\text{check}_V(C, (x, w), T)$.
6. P and V run $\text{LigeroProve}(C', (x, w'), T', \text{com})$ and $\text{LigeroVerify}(C', x, T', \text{com})$ respectively; V outputs the result.

Remarks The prover in line 2 commits to $|w| + \ell$ elements, which can be substantially smaller than the $|C|$ elements that the Ligero prover must commit to if it is applied directly to C . The Ligero commitment involves relatively expensive Reed-Solomon encoding over

$\mathbb{F}$. This improvement is especially important when $\mathbb{F}$ does not natively have suitable roots of unity, as is the case for the ECDSA circuit.

The Ligero prover runs on circuit C' which decrypts the transcript (via the one-time pad, i.e., using one field operation per element of T') and runs the check_V circuit. Roughly, C' performs a small number of field operations and then computes one dot product with (x, w) . Moreover, this dot product occurs between the hidden witness and a publicly computable challenge vector derived from the random challenges sent by the verifier. This public vector also has a *tensor-form* that is induced by the Verify protocol.

The main detail concerns how to “commit” to the transcript of the VC protocol: we use a one-time pad encryption in a way that allows an efficient Ligero proof about the check_V verifier. Namely, the one-time pad commitment creates linear Ligero constraints that are efficient to process. Our formulation only requires Ligero to handle d quadratic constraints where d is the depth of the original circuit (typically a small integer < 20), whereas the original circuit may contain thousands of multiplications.

In more detail, the verifier operations in sumcheck reduce to two cases. Recall that the verifier holds a “left-hand side” or claim $c \in \mathbb{F}$, which represents the summation of a multilinear-extension polynomial over the hyper-cube at a random point. As the sumcheck loop begins, the prover responds with a low-degree polynomial p of degree 2 expressed in the first variable of the hypercube. The first case of the check is to ensure that $p(0) + p(1) = c$, that is, to verify that the polynomial p aligns with the main claim of the sumcheck. Next, the verifier samples a random $r \in \mathbb{F}$ and evaluates $p(r)$ to define the next left-hand side claim. For our purposes, the evaluation at r can be written as the dot-product $\langle \lambda_r, (p(0), p(1), p(2)) \rangle$ for the Lagrange interpolation values λ_r . This loop proceeds over all of the variables of the sumcheck. Recall that our sumcheck (eliding the copy variables) is of the form $q(l, r) \cdot w(l) \cdot w(r)$ where q represents the quad-form wiring predicate. After all ℓ variables are processed, the verifier is left with an identity of the form $\langle \lambda_{r_\ell}, p_\ell \rangle = q(L, R) \cdot w(L) \cdot w(R)$ where r_ℓ was the last random challenge, the capital L, R represent vectors of all ℓ challenges, and p_ℓ is the last polynomial sent by the prover.

In our protocol, the verifier receives a padded, or encrypted form of each sumcheck message. For example, instead of receiving the polynomial in the form of evaluations $\{p(i)\}_{0,1,2}$, it instead receives $\{p(i) + \text{pad}_i\}_{0,1,2}$ with different pads used for each polynomial. In the last step, instead of receiving $w(L)$ and $w(R)$, the verifier receives $w(L) + \text{pad}_L$ and $w(R) + \text{pad}_R$. These $\{\text{pad}_i\}$, pad_L , and pad_R values are committed beforehand.

Because the verifier cannot directly check the encrypted sumcheck messages, it instead uses Ligero constraints on the committed pad values to ensure the same. In the first step, the verifier can add the *linear* Ligero constraint that $\text{pad}_0 + \text{pad}_1$ is equal to the values $(p(0) + \text{pad}_0) + (p(1) + \text{pad}_1) - c$ which are public values included in the encrypted transcript. This check ensures that $c = p(0) + p(1)$. In the next step, $p(r)$ can again be defined by adding constraints on the public padded values $\{p(i) + \text{pad}_i\}_{i=0,1,2}$ and the Lagrange coefficients for evaluation at r . In this way, the remaining polynomial constraints can be expressed via linear constraints on the pad values in the commitment.

The final check that $\langle \lambda_{r_\ell}, p_\ell \rangle = q(L, R) \cdot w(L) \cdot w(R)$ requires adding linear and quadratic Ligero constraints on the pad. Expanding public terms yields two relations:

$$\begin{aligned}
 (w(L) + \text{pad}_L) \cdot (w(R) + \text{pad}_R) &= w(L) \cdot w(R) + w(L)\text{pad}_R + w(R)\text{pad}_L + \text{pad}_L \cdot \text{pad}_R \\
 &= w(L) \cdot w(R) + w(L)\text{pad}_R + w(R)\text{pad}_L + \text{pad}_L \cdot \text{pad}_R \\
 &\quad + 2\text{pad}_L \cdot \text{pad}_R - 2\text{pad}_L \cdot \text{pad}_R \\
 &= w(L) \cdot w(R) + (w(L) + \text{pad}_L)\text{pad}_R \\
 &\quad + (w(R) + \text{pad}_R)\text{pad}_L - \text{pad}_L \cdot \text{pad}_R \\
 \langle \lambda, p_\ell + \text{pad}_\ell \rangle &= \langle \lambda, p_\ell \rangle + \langle \lambda, \text{pad}_\ell \rangle
 \end{aligned}$$

Let $Q = q(L, R)$, which can be computed by the verifier. Combining, and re-arranging:

Claim. $\langle \lambda_{r_\ell}, p_\ell \rangle = Q \cdot w(L) \cdot w(R)$ if and only if

$$\begin{aligned} \langle \lambda_{r_\ell}, p_\ell + \text{pad}_\ell \rangle - Q(w(L) + \text{pad}_L)(w(R) + \text{pad}_R) &= \langle \lambda_{r_\ell}, \text{pad}_\ell \rangle - Q(w(L) + \text{pad}_L)\text{pad}_R \\ &\quad - Q(w(R) + \text{pad}_R)\text{pad}_L \\ &\quad + Q \cdot \text{pad}_L \cdot \text{pad}_R \end{aligned}$$

The left hand-side of the claim includes only public values from the transcript, and the right-hand side consists of linear constraints on pad , pad_L , and pad_R and a quadratic constraint on $\text{pad}_L \cdot \text{pad}_R$.

To reduce the size of the transcript, it is well-known that $p(1)$ can be removed, and the verifier can implicitly infer its value as $c - p(0)$ for the purpose of interpolating at $p(r)$. This observation can be incorporated into the constraints generated above; in particular, the verifier can maintain a symbolic relationship on each $p(r)$, and deduce a single constraint that involves all pad variables for the layer, pad_L , and pad_R .

Theorem 6. *If C is a depth- d and width- w circuit over $\mathbb{F}$, and if Ligerio is a zero-knowledge protocol with adaptive knowledge soundness error $\epsilon_{FSK}(|C|, T, \lambda)$ and statistical zero-knowledge error $z'(C, \lambda)$, then Protocol 2.5 is an honest-verifier zero-knowledge protocol with completeness, knowledge soundness error bounded by $O(\eta(|C|, |\mathbb{F}|) + d \cdot \log(w)/|\mathbb{F}|)$ and zero-knowledge error bounded by $z'(C, \lambda)$.*

Proof. (Sketch) The completeness property follows by inspection.

To argue the claim on knowledge soundness error, it remains to analyze an extractor algorithm E' for the protocol. The extractor $E'^{P^*}(C, x)$ for this system receives the circuit C , the public input x , and oracle access to a prover P^* and performs the following steps in order to output a witness w for the statement $C(x, w) = 1$. Extractor E' runs the protocol with P^* until the beginning of step 6 in order to produce the values com and T' . It then runs the Ligerio extractor algorithm on the circuit C' , com , T' and x and uses the state of P^* at the beginning of step 6 as an oracle in order to simulate the Ligerio prover. If the Ligerio extractor fails, then E' also fails. If the Ligerio extractor succeeds by outputting a witness w' , then E' parses w' into (w, pad) , and then uses pad to decrypt the transcript $T \leftarrow T' \oplus \text{pad}$ (abusing notation). E' then runs the sumcheck verifier on C, x, T . If the sumcheck verifier succeeds, then E' verifies that $C(x, w)$ succeeds, and if so, outputs w . If either the sumcheck verifier fails, or the circuit check fails, then E' fails.

By construction, if E' outputs a value, it is guaranteed to be a valid witness because the final step of E' explicitly verifies that $C(x, w) = 1$. Without loss of generality, we assume that the Ligerio extractor behaves similarly, incurring all of its knowledge soundness error only when it fails to output a witness. Thus, bounding the error of E' (by union bound) only requires analyzing three failure cases. The first occurs if the Ligerio extractor fails. This is bounded by the knowledge soundness error given in Theorem 4. The second occurs if the Ligerio extractor outputs a witness, but the witness does not satisfy the sumcheck verifier. This probability is 0 by the above-mentioned soundness property of Ligerio, and the construction of the check_V circuit. Finally, it is possible that the Ligerio extractor outputs a witness w that satisfies the sumcheck verifier, but nonetheless does not satisfy the circuit: $C(x, w) = 0$. In this case, transcript T represents a soundness failure for the sumcheck verifier. The statistical soundness gap of the verify protocol from Theorem 1 bounds this value. Altogether, the union bound of these 3 events yields the bound on soundness error given in the theorem statement.

The zero-knowledge property follows directly from the Ligerio simulator run on the augmented circuit C' . Specifically, the simulator for Protocol 2.5 proceeds as follows. In the first step, it samples a random encrypted transcript T' . It then invokes the Ligerio simulator on input statement (C', x, T') , which produces an output com, π . Finally, the simulator outputs com, T', π .

To analyze the simulator, observe that the distribution of T' produced by the simulator is identical to that produced by an honest prover, as T' is a one-time pad encryption in both cases. Since the statements (C', x, T') are identically distributed, we can rely directly on the zero-knowledge property of Ligerio. Specifically, Theorem 4 guarantees an honest-verifier simulator for Ligerio with zero-knowledge error bounded by $z'(C, \lambda)$. Consequently, this value also bounds the probability of distinguishing the simulated output from an honest execution, matching the claim. $\square$

2.4 Applying the Fiat-Shamir Transform

We apply the Fiat-Shamir transform on our basic protocol to construct a non-interactive protocol, and argue soundness by applying the round-by-round Soundness lemma presented by Canetti et al [CCH⁺18].

Theorem 7 ([CCH⁺18]). *Suppose that $\Pi = (P, V)$ is a $2r$ -message public-coin interactive proof for a language L with perfect completeness, $\text{poly} \log(n)$ total bits of prover-to-verifier communication, and round-by-round soundness with a corresponding state function State . Let X_n denote the set of partial transcripts (including the input and all messages sent) and let Y_n denote the set of verifier messages when Π is executed on an input of length n . If a hash family $H = \{H_n : X_n \rightarrow Y_n\}$ is R_{State} -correlation intractable and evaluable in time $\tilde{O}(n)$, then there exists $(\text{Gen}, \tilde{P}, \tilde{V})$ that constitute an adaptively sound publicly verifiable argument for L .*

Canetti et al [CCH⁺18] argue that sumcheck is round-by-round sound with $O(d/|\mathbb{F}|)$. Ligerio is also a round-by-round sound with $O(\eta(C')/|\mathbb{F}|)$.

It remains to argue that the implementation of the random oracle is correlation-intractable with respect to the round-by-round state function for sumcheck and Ligerio. In practice, whether an implementation of the random oracle satisfies this correlation-intractability property becomes an implicit assumption. Towards that, we adapt best practices in designing the oracle implementation. To avoid issues of adaptive soundness, our protocol selects the random oracle *after* picking the circuit to prove. We also choose a random oracle that has higher circuit depth and requires more gates to compute than the circuit C . Furthermore, the size of the transcript used as a query to the oracle to generate the Verifier's challenges are guaranteed to be larger than C . We incorporate the Prover's first message, an identifier for the circuit, the input and output of the theorem statement as well as a string of 0-bytes that is larger than $|C|$ into the prefix of the transcript. These choices are easy to implement and add very little processing time to the protocol. On the other hand, they seemingly avoid attacks such as the one suggested by Khovratovich, Rothblum, and Soukhanov [KRS25] against correlation-intractability in which the random oracle is computed within the ZK protocol thereby allowing the output of the circuit to be related to the verifier's challenge. As an additional property, we ensure that each query to the random oracle can be uniquely mapped into a protocol transcript. To facilitate this property, the type and length of each message is incorporated into the query string.

3 Optimization

3.1 Reed-Solomon encoding over prime fields

The Ligerio polynomial commitment specifically requires Reed-Solomon codes, as opposed to generic error-correcting codes. Ligerio interprets the message (an array of field elements) as the evaluations of a low-degree polynomial at certain points, and while the choice of the points does not matter for soundness, it does affect performance. We describe our choices for prime fields $\mathbb{F}_q$ here and binary fields in Section 3.2.

A natural set of evaluation points would be ω^i , for a root of unity ω , so that FFT-based methods can be used for interpolation. However, this choice leads to complicated interactions between the factorization of $(q-1)$, the length of the codeword, and the efficiency of the interpolation. In particular, $\mathbb{F}_{P256}$ where P256 is the NIST prime $P256 = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$ has no usable roots of unity.

Instead, we define a *message* to consist of the evaluations of a polynomial $p(x)$ of degree at most d at points $x \in \{\mathcal{F}_q(i) : 0 \leq i \leq d\}$, where $\mathcal{F}_q(i) = i \bmod q$ is the natural injection of integer i into the field $\mathbb{F}_q$. Similarly, a *codeword* of length m consists of the m evaluations $\{p(\mathcal{F}_q(i)) : 0 \leq i < m\}$. These definitions are well-formed for fields of sufficiently large characteristic, and the only remaining problem is how to compute the polynomial interpolation.

Interpolation via Convolution Given the evaluations $p(i)$ at integer points $0 \leq i \leq d$, we want to compute $p(k)$ for all integers k in some range. As in [CMT12, Section 2.1], our general strategy is to reduce polynomial interpolation to convolution, and then apply the most convenient convolution algorithm.

Specifically, we use the following identity, which holds over the integers: For integer $k > d$,

$$p(k) = (-1)^d \cdot (k-d) \binom{k}{d} \cdot \sum_{0 \leq i \leq d} \left(\frac{1}{k-i} \right) \cdot (-1)^i \binom{d}{i} p(i). \quad (2)$$

Except for scaling by constants, Equation (2) is indeed a (linear) convolution: Multiply the input $p(i)$ elementwise by the constants $(-1)^i \binom{d}{i}$, convolve with the kernel $(k-i)^{-1}$, and divide elementwise by the constants $(-1)^d \cdot (k-d) \binom{k}{d}$ yielding $p(k)$. Equation (2) also holds $(\bmod q)$ when $k < q$. We now present a proof of this folklore identity.

Lemma 1. (Lagrange interpolation) Let $p(k)$ be a polynomial of degree at most d over a field. For all k we have

$$p(k) = \sum_{0 \leq i \leq d} \binom{k}{i} \binom{d-k}{d-i} p(i). \quad (3)$$

Proof. Let $q(k)$ be the right-hand side of Equation (3). We want to prove that $p = q$.

The binomial coefficient $\binom{x}{r}$ is a polynomial in x of degree at most r . Thus $q(k)$ is a polynomial of degree at most d . We now prove that $p(k) = q(k)$ for $0 \leq k \leq d$, and therefore that the two polynomials are the same.

Assume $0 \leq k \leq d$. If $i = k$, the i -th term of the sum is $p(k)$. If $i > k$, we have $\binom{k}{i} = 0$, and thus the i -th term of the sum is 0. If $i < k$, we have $\binom{d-k}{d-i} = 0$, and thus i -th term of the sum is also 0. Thus $q(k) = p(k)$ as desired. $\square$

Lemma 2. (Lagrange interpolation via convolution.) Let $p(k)$ be a polynomial of degree at most d over a field. For $k > d$, we have

$$p(k) = (-1)^d \cdot (k-d) \binom{k}{d} \cdot \sum_{0 \leq i \leq d} \left(\frac{1}{k-i} \right) \cdot (-1)^i \binom{d}{i} p(i). \quad (2)$$

Proof. The proof is a straightforward manipulation of Equation (3) to separate the terms that depend on k from those that depend on i and on $k-i$.

We use the well-known binomial negation of the upper index

$$\binom{-r}{s} = (-1)^s \binom{r+s-1}{s}$$

to make the upper index non-negative in $\binom{d-k}{d-i}$, as follows.

$$\binom{k}{i} \binom{d-k}{d-i} = (-1)^{d-i} \binom{k}{i} \binom{k-i-1}{d-i}.$$

Expanding the binomial coefficients on the right-hand side into factorials and simplifying, we have

$$\begin{aligned} &= (-1)^{d-i} \frac{k!}{i!(k-i)!} \cdot \frac{(k-i-1)!}{(d-i)!(k-d-1)!} \\ &= \frac{(-1)^{d-i}}{k-i} \cdot \frac{k!}{i!(d-i)!(k-d-1)!} \\ &= \frac{(-1)^{d-i}}{k-i} \cdot \frac{1}{d!} \cdot \binom{d}{i} \cdot (k-d) \cdot d! \cdot \binom{k}{d} \\ &= \frac{(-1)^{d-i}}{k-i} \cdot \binom{d}{i} \cdot (k-d) \cdot \binom{k}{d} \\ &= (-1)^d \cdot \frac{1}{k-i} \cdot (-1)^i \binom{d}{i} \cdot (k-d) \binom{k}{d}, \end{aligned}$$

from which Equation (2) follows. $\square$

Remark 1. The equality

$$(k-d) \binom{k}{d} = k \binom{k-1}{d}$$

offers a way to compute $\binom{k}{d}$ in linear time for k ranging over an interval.

Alternatively, Equation (2) gives the explicit form of a special case of the matrix factorization $V_0^{-1}V_1 = D_0CD_1$, where V_i are Vandermonde, C is Cauchy, and D_i are diagonal [Gow92, Theorem 1].

Many alternatives exist to compute convolutions over finite fields. If the order of the multiplicative group of the field factors into small primes, then the convolution can be computed in $O(n \log n)$ field operations via some form of FFT. Without breaking the field abstraction, Nussbaumer's algorithm [Nus80] computes a convolution over arbitrary fields using $O(n \log n \log \log n)$ field operations. (See also [Knu97, Exercise 4.6.4-59].) Alternatively, one can compute convolutions in a prime field $\mathbb{F}_p$ by computing the convolution over integers and reducing the answer mod p . In turn, the integer convolution can be computed in a larger FFT-friendly prime field $\mathbb{F}_q$. Choosing $q \geq np^2$ is sufficient for representing the integer result exactly.

In our case, the ECDSA signature is standardized over a small set of elliptic curves such as P256, P384, P521. Each curve is defined over a specific *base* finite field $\mathbb{F}_q$ for prime q which is not FFT friendly. However, every odd prime q is of the form $q = r \cdot 2^\ell - 1$, and the quadratic extension $\mathbb{F}_{q^2}$ contains roots of unity of order 2^ℓ , because the order of the multiplicative group is $q^2 - 1 = (q+1)(q-1)$ and 2^ℓ divides $(q+1) = r \cdot 2^\ell$. While this fact is not useful in the trivial case $\ell = 0$, in the specific case of the ECDSA primes one can pick ℓ large enough that the FFT over the extension field is viable. Zhang et al. [ZXZS19] make the same observation in the case q is a Mersenne prime $q = 2^\ell - 1$. Thus, for the ECDSA primes, the convolution is computed in the base field via two FFTs in the quadratic extension, with precomputation of the transform of the convolution kernel. Because the inputs to the convolution are in the base field, we use a radix-4 variant of the real-valued FFT [SJB87] for efficiency, as opposed to lifting the input into the extension field and computing a full extension-field FFT.

3.2 Reed-Solomon encoding over binary fields

In fields of characteristic 2, $1 + 1 = 0$ and the choice of interpolation points in Section 3.1 does not apply. Instead, as in [LCH14, DP24], a more complex choice is needed.

$\text{GF}(2^k)$ forms a vector space of dimension k over $\text{GF}(2)$. Fix once and for all a basis β_i of this vector space, where $0 \leq i < k$. (See Section 3.3 for a concrete choice of basis.) Writing j_i for the i -th bit of the binary representation of j , that is, $j = \sum_{0 \leq i < k} j_i 2^i$ and $j_i \in \{0, 1\}$, we inject integer j into a field element $\mathcal{F}(j)$ by interpreting the bits of j as coordinates in terms of the basis:

$$\mathcal{F}(j) = \sum_{0 \leq i < k} j_i \beta_i. \quad (4)$$

Then, as in Section 3.1, we define the message to be encoded to consist of the evaluations of a polynomial $p(x)$ of degree at most d at the $d + 1$ points $x \in \{\mathcal{F}(i) : 0 \leq i \leq d\}$. Similarly, a codeword of length m consists of the m evaluations $\{p(\mathcal{F}(i)) : 0 \leq i < m\}$.

The particular choice of the injection $\mathcal{F}(j)$, as opposed for example to $\mathcal{F}(j) = \omega^j$ for some appropriate root of unity ω , is motivated by the existence of the *additive FFT*, which solves the interpolation problem in $O(m \log d)$ operations.

We use the additive FFT first presented by Lin, Chung, and Han [LCH14], which builds upon previous algorithms with higher asymptotic complexity. Lin et al. define a *novel polynomial basis* for polynomials as an alternative to the usual monomial basis x^i , and give an algorithm for evaluating a degree- $(d - 1)$ polynomial at all d points in a subspace, for $d = 2^\ell$, and for polynomials expressed in the novel basis. The algorithm can be executed backwards to reconstruct the coefficients in the novel basis given the evaluation points. See [LANHC16, LCK⁺18] for alternative presentations of the same idea.

While the algorithm in [LCH14] is recursive, Diamond and Posen [DP24, Algorithm 2] reformulate the algorithm as a triply-nested loop, resembling common implementations of the Cooley-Tukey FFT.² In addition, Diamond and Posen extend the algorithm to obtain $2^{\ell+\mathcal{R}}$ evaluations of a polynomial of degree less than 2^ℓ , whereas [LCH14] evaluates 2^ℓ points at an arbitrary coset of a subspace.

Our contribution is to show how to avoid restrictions such as d and m being powers of two, even though the additive FFT algorithms only work for 2^ℓ sizes. For the evaluation of a polynomial of arbitrary degree d , we use the triply-nested loop formulation of Diamond and Posen with $\mathcal{R} = 0$, augmented with the original coset parameter from [LCH14]. We pad the array of coefficients with zeros to reach size 2^ℓ , and evaluate sufficiently many cosets to obtain the desired m evaluations, possibly discarding unneeded points.

The reconstruction of the polynomial coefficients in the novel basis given $d + 1$ evaluations is more challenging when $d + 1$ is not a power of two. We cannot simply pad the evaluations with $2^\ell - (d + 1)$ zeroes and apply the inverse additive FFT, since this process would produce a polynomial of degree $2^\ell - 1$ and not d . To solve this problem, we adapt the inverse *truncated FFT* of van der Hoeven [vdH04] to work with the additive FFT algorithm. The truncated FFT computes a subset of the outputs of the FFT, and van der Hoeven [vdH04] presents an elegant algorithm for inverting the process. While the truncated FFT literature is mostly concerned with saving field operations or avoiding storing implicit zeroes, we prefer to look at the algorithm as a bidirectional FFT, as follows. Consider a linear transformation

$$\begin{bmatrix} Y_0 \\ Y_1 \end{bmatrix} = A \begin{bmatrix} X_0 \\ X_1 \end{bmatrix},$$

²The Cooley-Tukey FFT requires a bit-reversal of either the input or the output, but the Diamond and Posen algorithm does not, so the two algorithms are not quite the same.

where X_i and Y_i are column vectors of n_i elements. For the concrete case where A is the Fourier matrix, or its additive variant, the forward FFT computes Y given X , and the inverse FFT computes X given Y . We call an algorithm that computes Y_1 and X_0 given Y_0 and X_1 a *bidirectional FFT*. Thus, a bidirectional FFT algorithm computes either the forward or inverse transform, as well as many combinations thereof.

Details of the additive FFT We follow the notation introduced by Diamond and Posen [DP24]. Let U_i be the span of the prefix $\{\beta_k : 0 \leq k < i\}$ of the basis. Define the *subspace vanishing polynomial* $W_i(x) = \prod_{u \in U_i} (x - u)$ and its normalized variant $\widehat{W}_i(x) = W_i(x) / W_i(\beta_i)$.

Polynomials W_i are *linearized*: $W_i(x + y) = W_i(x) + W_i(y)$. See [vzGG96, Lemma 2.3], where this fact is stated without proof. See Mateer’s thesis [Mat08, Theorem 15] for a proof. Linearization is crucial for the efficiency of the additive FFT, and it also allows for the efficient computation of $W_i(x)$: If $x = \sum_k x_k \beta_k$ in terms of the basis, then $W_i(x) = \sum_k x_k W_i(\beta_k)$.

Moreover, $W_{i+1}(x) = W_i(x)W_i(x + \beta_i)$ because $U_{i+1} = U_i \cup (\beta_i + U_i)$, and thus $W_{i+1}(x) = W_i(x)(W_i(x) + W_i(\beta_i))$ holds by linearity. Together with the base case $W_0(x) = x$, these two properties allow for efficient computation of $W_i(\beta_j)$ and $\widehat{W}_i(\beta_j)$ for all necessary i and j . We precompute and store $\widehat{W}_i(\beta_j)$ in an array $\widehat{W}[i][j]$.

The additive FFT [LCH14] evaluates a polynomial

$$p(x) = \sum_{0 \leq k < 2^i} c_k X_k(x)$$

for all x in a coset $U_i + \alpha$, given the *coefficients* c_k of the polynomial’s expansion in terms of the *novel polynomial basis* $X_k(x)$. Running the algorithm backwards (the inverse FFT) computes the coefficients from the evaluations. Lin et al. [LCH14, Equation 6] give an explicit formula for the basis $X_k(x)$, but the precise form of $X_k(x)$ does not really matter for interpolation as long as $X_k(x)$ is a polynomial of degree k . That is, one can start from the evaluations over U_i , compute the coefficients, and evaluate the polynomial over a coset $U_i + \alpha$ to obtain new evaluations, and nowhere does one have to know the exact form of the basis.³

Algorithm 1 shows pseudo-code for our implementation of the additive FFT, largely lifted from [DP24, Algorithm 2], and of its inverse. We show procedure TWIDDLE, which computes the “twiddle factors,” as presented in the literature, but in the actual implementation we compute all twiddle factors at the same time via the recurrence $\text{TWIDDLE}(i, u + 2^k) = \widehat{W}[i][k] + \text{TWIDDLE}(i, u)$, which yields an algorithm for doubling the size of an array of twiddle factors in linear time.

Details of the bidirectional FFT Ignoring bit-reversal permutations, the computation graph of the additive FFT is the same as the one of the ordinary Cooley-Tukey FFT. (See Figure 1.) Given n input coefficients $C[\cdot]$ in the novel polynomial basis, a loop of elementary *butterfly* operations decomposes the input even $C_e[\cdot]$ and odd $C_o[\cdot]$ arrays, to which the transform is applied recursively yielding the evaluation array $E[\cdot]$.

In the bidirectional FFT setup, the inputs consist of evaluations $E[i]$ for a prefix $0 \leq i < k$ of E , and coefficients $C[i]$ for a suffix $k \leq i < n$ of C . The goal is to compute the remaining coefficients and evaluations. To this end, we adapt the discussion from [vdH05, Section 6] to the case of additive FFT.

³The situation is somewhat analogous to Newton finite-difference methods of interpolation, where one starts with the evaluations and computes a bunch of additions and subtractions, obtaining evaluations at additional points. In one sense, some intermediate values can be interpreted as the expansion of the polynomial in terms of the Newton basis, but in another sense, one can view the method as an algorithm for interpolation without any mention of the Newton basis.

Algorithm 1 Additive FFT [DP24, Algorithm 2]

```

procedure FFT( $\ell, \alpha, B[]$ )
  ▷ Require: Array  $B$  of size  $2^\ell$  containing the coefficients of the expansion  $p(x) =$ 
     $\sum_{0 \leq k < 2^\ell} B[k] X_k(x)$  in terms of the novel polynomial basis  $X_k(x)$ .
  ▷ Ensure:  $B[k] \leftarrow p(\mathcal{F}(k + \alpha))$ , where  $\mathcal{F}(\cdot)$  is defined in Equation (4).
  for all  $i : 0 \leq i < \ell$  in descending order of  $i$  do
     $s \leftarrow 2^i$ 
    for all  $u : 0 \leq 2s \cdot u < 2^\ell$  do
       $t \leftarrow \text{TWIDDLE}(i, 2s \cdot u + \alpha)$ 
      for all  $v : 0 \leq v < s$  do
        BUTTERFLY-FWD( $B, 2s \cdot u + v, s, t$ )

procedure IFFT( $\ell, \alpha, B[]$ )
  ▷ Inverse of procedure FFT.
  for all  $i : 0 \leq i < \ell$  in ascending order of  $i$  do
     $s \leftarrow 2^i$ 
    for all  $u : 0 \leq 2s \cdot u < 2^\ell$  do
       $t \leftarrow \text{TWIDDLE}(i, 2s \cdot u + \alpha)$ 
      for all  $v : 0 \leq v < s$  do
        BUTTERFLY-BWD( $B, 2s \cdot u + v, s, t$ )

procedure TWIDDLE( $i, u$ )
  Let  $u_k$  denote the  $k$ -th bit of  $u$ , i.e.,  $u = \sum_k u_k 2^k$ .
  return  $\sum_k u_k \cdot \widehat{W}[i][k]$ 

procedure BUTTERFLY-FWD( $B[], w, s, t$ )
   $B[w] \leftarrow B[w] + t \cdot B[w + s]$ 
   $B[w + s] \leftarrow B[w + s] + B[w]$ 

procedure BUTTERFLY-BWD( $B[], w, s, t$ )
   $B[w + s] \leftarrow B[w + s] - B[w]$ 
   $B[w] \leftarrow B[w] - t \cdot B[w + s]$ 

procedure BUTTERFLY-DIAG( $B[], w, s, t$ )
   $(B[w], B[w + s]) \leftarrow (B[w] - t \cdot B[w + s], B[w] + B[w + s])$ 

```

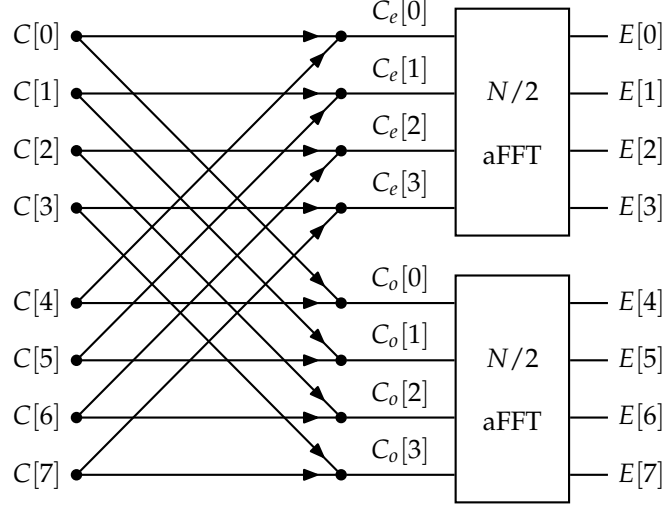


Figure 1: Flow graph of the ordinary additive FFT, which is the same as the one for the Cooley-Tukey FFT if one ignores bit-reversal permutations. The “butterflies” with two inputs and two outputs denote that the output is some 2×2 linear transformation of the input. Lin and others [LCH14] draw a slightly different diagram factoring the linear transformation into triangular matrices, but for our purposes this level of detail is not important.

Consider the elementary *forward butterfly* in procedure BUTTERFLY-FWD in Algorithm 1. Let $(a, b) = (B[w], B[w + s])$ before applying the butterfly operation, and $(c, d) = (B[w], B[w + s])$ after applying the operation. Then the butterfly computes

$$\begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} 1 & t \\ 1 & t+1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}. \quad (5)$$

Similarly the *backward butterfly* operation in BUTTERFLY-BWD computes the inverse transformation

$$\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} t+1 & -t \\ -1 & 1 \end{bmatrix} \begin{bmatrix} c \\ d \end{bmatrix}. \quad (6)$$

The key insight [vdH05, Equation (7)] of the bidirectional FFT algorithm is the *diagonal butterfly* (BUTTERFLY-DIAG), which proceeds forward on b and backwards on c :

$$\begin{bmatrix} a \\ d \end{bmatrix} = \begin{bmatrix} -t & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} b \\ c \end{bmatrix}. \quad (7)$$

The symmetric case of computing (b, c) from (a, d) is also viable for the ordinary Cooley-Tukey FFT [vdH05, Equation (8)]. However, the corresponding equation for the additive FFT

$$\begin{bmatrix} b \\ c \end{bmatrix} = (1+t)^{-1} \cdot \begin{bmatrix} -1 & 1 \\ 1 & t \end{bmatrix} \begin{bmatrix} a \\ d \end{bmatrix} \quad (8)$$

requires $(1+t)$ to be invertible, which is not guaranteed in our case. This is the reason why we chose the convention that a prefix of the evaluations are known, as opposed to a prefix of the coefficients. With this choice, we only need (7) but not (8).

Consider again Figure 1, and assume that $k \geq n/2$. Thus, the entire output of the top $N/2$ -FFT block is known, the entire $C_e[\cdot]$ can be obtained from the inverse FFT of the known prefix of $E[\cdot]$. Given $C_e[\cdot]$ and the known suffix $C[i]$ for $i \geq n - k$, diagonal

Algorithm 2 Bidirectional FFT, adapted from [vdH05, §6]

```

procedure BIDIRECTIONAL-FFT( $i, \alpha, k, B[]$ )
  ▷ Require: Array  $B$  of length  $n = 2^i$ , whose first  $k$  elements are evaluations of a polynomial
  ▷ and the rest are coefficients of the same polynomial in the novel polynomial basis.      ◁
  ▷ Ensure: Overwrite  $B$  with an array whose first  $k$  elements are coefficients and the rest
  ▷ are evaluations.      ◁
  if  $i > 0$  then
     $i \leftarrow i - 1$ 
     $s \leftarrow 2^i$ 
     $t \leftarrow \text{TWIDDLE}(i, \alpha)$ 
    if  $k < s$  then
      for all  $v : k \leq v < s$  do
        [ BUTTERFLY-FWD( $B, v, s, t$ )
        BIDIRECTIONAL-FFT( $i, \alpha, k, B$ )
      for all  $v : 0 \leq v < k$  do
        [ BUTTERFLY-DIAG( $B, v, s, t$ )
        FFT( $i, \alpha + s, B[s:]$ )
    else
      IFFT( $i, \alpha, B$ )
      for all  $v : k - s \leq v < s$  do
        [ BUTTERFLY-DIAG( $B, v, s, t$ )
        BIDIRECTIONAL-FFT( $i, \alpha + s, k - s, B[s:]$ )
      for all  $v : 0 \leq v < k - s$  do
        [ BUTTERFLY-BWD( $B, v, s, t$ )

```

butterflies compute $C_o[i]$ for $i \geq k - n/2$. The result is a bidirectional FFT problem of size $n/2$: a suffix of length $n/2 - (k - n/2)$ of C_o and a prefix of length $k - n/2$ of the bottom half of E . This bidirectional subproblem is solved recursively, thus computing the missing elements of E and C_o . Given the entire C_e and C_o , the missing C 's are computed via backward butterflies.

Symmetric considerations apply to the case $k < n/2$. See Algorithm 2 for complete pseudo-code of the bidirectional FFT algorithm.

Even though the FFT algorithm reduces a problem of size n to two subproblems of size $n/2$, the bidirectional FFT algorithm produces one bidirectional problem of size $n/2$ and another problem of size $n/2$ which is either a fully forward or a fully backward FFT. Thus, assuming an optimized implementation of the latter transform, the overhead of the recursion is minimal: $O(\log n)$ recursive calls and $O(\log n)$ computations of twiddle factors. In practice, the performance of the bidirectional FFT is essentially the same as that of the fully forward or fully backward FFT.⁴

Details of Reed-Solomon encoding Our original interpolation problem takes $d + 1$ evaluations of a degree- d polynomial $p(\mathcal{F}(i))$ for $0 \leq i \leq d$ as input, and must compute $p(\mathcal{F}(i))$ for $0 \leq i < m$, where d and m are arbitrary (i.e., not necessarily powers of 2). Let ℓ be minimal such that $d < 2^\ell$. Run the bidirectional FFT algorithm with $n = 2^\ell$ and $k = d + 1$, assuming that the first k coefficients are unknown and the remaining are zero. The bidirectional FFT produces the first missing $(n - k)$ evaluations, as well as the full array of k coefficients. Then the forward FFT can be run on as many cosets as needed to produce m evaluations.

⁴With some care, the recursion can be removed entirely, but we found the recursion-free algorithm needlessly confusing.

3.3 Reduction of the Ligero proof size

Soundness requires using large fields such as $\text{GF}(2^{128})$. We implement $\text{GF}(2^{128})$ as $\text{GF}(2)[x]/(Q(x))$ where $Q(x) = x^{128} + x^7 + x^2 + x + 1$. With this choice of $Q(x)$, x is a generator of the multiplicative group of the field.

Almost all messages in our system consist of witness *bits* in $\{0, 1\}$ that are packed into arrays of size less than 2^{14} . Recalling Reed-Solomon encoding from Section 3.2, if the message is an array of elements in some subfield of $\text{GF}(2^{128})$, and if the evaluation points are also elements in the same subfield, then the entire codeword consists of subfield elements and it can be represented compactly. In order to ensure that all evaluation points are in the subfield, it suffices to choose β_i as a basis of the subfield rather than the entire field.

Thus, we choose $\text{GF}(2^{16})$ as the subfield of $\text{GF}(2^{128})$, with $g = x^{(2^{128}-1)/(2^{16}-1)}$ as the generator of $\text{GF}(2^{16})$, and $\beta_i = g^i$ for $0 \leq i < 16$ as the basis of the subfield. This allows us to encode elements in our commitment scheme with 16-bits instead of 128. Whenever we have a choice, we inject witness bits and small integers into the subfield via Equation (4).

This optimization introduces the problem of determining whether a general field element $u \in \text{GF}(2)[x]/(Q(x))$ belongs to the subfield, and if so, to determine u 's representation $u = \sum_{0 \leq i < 16} u_i \beta_i$ in terms of the basis β_i . We solve this problem via standard Gaussian elimination. First, interpret elements of $\text{GF}(2^{128})$ as row vectors of 128 elements of $\text{GF}(2)$ with respect to the monomial basis x^j ; this convention allows two row vectors to be added via a single 128-bit xor instruction. We then form a matrix B whose i -th row consists of the row vector β_i . With these conventions, we need to solve the rectangular system $xB = u$ for x , which we do via preliminary LU factorization that reduces B to row-echelon form, followed by standard solution of triangular systems. The solution costs $O(n)$ row operations for $n = 16$, which is efficient since row operations take a single wide xor instruction. In our system, the time spent on projections onto the subfield is negligible.

In our implementation, the subfield optimization only reduces proof size, specifically during I/O time to serialize and deserialize field elements. We perform all arithmetic operations in the full field. Diamond and Posen [DP24] argue for a tower representation of the field which makes subfields explicit, and which can exploit subfield arithmetic when possible.

3.4 Input consistency over different Fields

The round-by-round soundness property requires that the sumcheck and verify protocols use a field of size at least 2^{128} . Ligero commitments require a subfield of size at least $O(\sqrt{|(x, w)|})$, where (x, w) is the size of the input to the circuit C . Thus, it is usually most efficient to run sumcheck over a field extension of a sufficiently large base field. However, the ECDSA circuit imposes a different constraint: namely that efficiently verifying elliptic curve operations over curve $P256$ requires the circuit to be defined over a specific 256-bit field.

Recall the ZK theorem (e, pk) which verifies that there exists a message m and signature (r, s) such that $e = \text{SHA256}(m)$, and (r, s) is an ECDSA signature on e under public key pk . A circuit that verifies this statement over $\mathbb{F}_{p^{256}}$ incurs the overhead of verifying the sha256 hash of m over a larger prime field than is necessary for soundness. Similarly, a circuit that verifies the full theorem over $\mathbb{F}_{2^{128}}$ incurs the $\approx 100x$ overhead of simulating the arithmetic for the elliptic curve.

A natural approach for higher efficiency is to perform the ECDSA verification of instance (e, pk) using circuit C_{sig} in the optimal $\mathbb{F}_{p^{256}}$ field, and perform the SHA verification of the instance (e, m) using circuit C_h over the optimal binary field. The concern is that a cheating prover may use 2 inconsistent values of e in the 2 different circuits in

two different fields. In this section, we describe a technique to ensure consistency of inputs when the verification of the theorem occurs using *two* or more proof fields, or more generally, two or more proof systems.

Our solution for input consistency is to augment each circuit in the two fields with a message authentication code (MAC) verification of the common input e . We use the information-theoretic $\text{MAC}_{a,b}(x) = ax + b$ where⁵ the arithmetic is performed over $\mathbb{F}_{2^{128}}$. In the ECDSA circuit, this requires simulating the arithmetic of $\mathbb{F}_{2^{128}}$ over the prime field $\mathbb{F}_{p^{256}}$. For small inputs, this computation incurs little overhead: each circuit is augmented to take as a public input $m_e = \text{MAC}_k(e)$, and as a private input, the key $k = (a, b)$, and each circuit internally computes the MAC on e and compares it with the public input.

A secondary issue relates to how the key k is chosen. Allowing the verifier to select k may break the zero-knowledge property; allowing the prover to select k breaks the soundness, since the prover can intentionally select a key that allows two different values of e to have the same MAC value. In our scheme, this key is jointly sampled by the Prover and Verifier. Specifically, the Prover first commits to its share $k_p = (a_p, b_p)$ as well as the secret input e . The verifier next randomly samples k_v and sends it to the Prover in the clear. The Prover then computes the key $k = k_p + k_v$, and then uses k to compute the mac on the shared input e , and then sends the MAC value m_e to the verifier. The verifier uses m_e as a public input to both circuit C_h and circuit C_{sig} . Both of these circuits include a check to verify that $m_e = \text{MAC}(e)$ using the Prover's private key share k_p and the Verifier's public input k_v .

Protocol 3.8. Two-fields ZK protocol (P_2, V_2)

Let circuit $C_1(x_1, w_1, w)$ be defined over field $\mathbb{F}_1$ and $C_2(x_2, w_2, w)$ defined over $\mathbb{F}_2$, and let $w \in \{0, 1\}^c$ be the common input to both that is embedded into the respective $\mathbb{F}_i$ in the natural way, and $x_1, w_1 \in \mathbb{F}_1^*$ and $x_2, w_2 \in \mathbb{F}_2^*$. Both parties hold (x_1, x_2) , and the prover additionally holds (w_1, w_2, w) .

- Define circuits $\hat{C}_i((x_i, m, k_v), (w_i, w, k_p))$ where $k_p, k_v \in \{0, 1\}^{256}$ as follows:
 - Assert $C_i(x_i, w_i, w) = 0$
 - Assert $\text{MAC}_k(w) = m$ where $k = k_p \oplus k_v$ with all computation performed in $\text{GF}(2^{128})$.
- P, V run the first 3 steps of Protocol 2.5 on circuit $\hat{C}_1$ and then on $\hat{C}_2$ with their respective inputs except with the following changes:
 3. P samples $k_p \leftarrow \text{GF}(2^{128})^2$ and then computes

$$\text{com}_i \leftarrow \text{LigeroCommit}((w_i, w, k_p, \text{pad}_i), \mathbb{F}_i)$$
 and sends to V for $\hat{C}_1$ and then for $\hat{C}_2$
 - (a) V samples $k_v \leftarrow \text{GF}(2^{128})^2$ and sends to P .
 - (b) P computes $m \leftarrow \text{MAC}_{k_v \oplus k_p}(w)$ and sends to V .
- Run step 4 of Protocol 2.5 for $\hat{C}_1$ with public input (x_1, m, k_v) and private input (w_1, w, k_p) and then the same for $\hat{C}_2$ with public input (x_2, m, k_v) and private input (w_2, w, k_p) to define transcripts $\hat{T}_1$ and $\hat{T}_2$.
- Run steps 5,6 for $\hat{C}_1$ and then the same for $\hat{C}_2$.

⁵When the common value e is known not to be 0, the MAC can be optimized to $a \cdot x$.

Theorem 9. *If C_1 and C_2 are circuits over fields $\mathbb{F}_1$ and $\mathbb{F}_2$, and if MAC_k is an information-theoretic message authentication code with statistical security λ_s , if Protocol 2.5 is a zero-knowledge protocol with adaptive knowledge soundness error $\eta(|C_1|, |C_2|)$, then Protocol 3.8 is a zero-knowledge protocol with knowledge soundness error $O(\eta(|C_1|, |C_2|) + \lambda_s)$.*

Proof. (Sketch) The modified protocol is a composition of Protocol 2.5. The completeness follows by inspection. It is more convenient to present a self-contained argument for zero-knowledge and soundness; the argument follows the one for Theorem 6.

To construct a knowledge-soundness extractor algorithm E for this composed protocol, run the extractor for C_1 and for C_2 sequentially against the prover P^* of Protocol 3.8. The only issue is that the extractor for C_i expects to receive only messages from the C_i instance. This syntactic issue is handled by simulating the honest public-coin verifier for P^* and filtering the appropriate messages for the extractor. If either extractor fails, then the composed protocol extractor fails. Suppose the extractor for C_1 outputs $(x_1, w_1, w, \text{pad}_1)$ and the second extractor outputs $(x_2, w_2, w', \text{pad}_2)$. If $w \neq w'$, then the composed extractor fails. Otherwise, output $(x_1, x_2, w_1, w, w_2, w)$ and halt.

Recall that the extractor for Protocol 2.5 either fails or outputs a witness that satisfies the circuit that defines its theorem statement. Thus, extractor E fails if either of the individual extractors fails or if the random key sampled for the MAC leads to a MAC collision on different input values. The second clause of this claim follows from the two constraints imposed on $\hat{C}_1$ and $\hat{C}_2$ above. Namely, because the circuits verify that $\text{MAC}_k(w) = m$ and $\text{MAC}_k(w') = m$ respectively, the condition that $w \neq w'$ only occurs when the MAC_k fails by a collision.

By a standard union bound argument over the failure probability of the individual extractors and the statistical soundness parameter of the MAC, the combined extractor succeeds with all but negligible probability. The running time of the composed extractor is bounded by a function of the running times for the individual extractors and the honest verifier strategy, and thus polynomial-time in the running time of a malicious prover oracle. In particular, the Ligerio extractor is straight-line extractable in the observable Random-oracle model, and thus there is no overhead to the running time in that context. Similarly, the round-by-round soundness of the protocol remains the same conditioned on the security of the MAC.

To argue zero-knowledge, consider the following simulator algorithm: Run steps 3a to sample a random k_v and instead of 3b, sample a random m . This now defines the public input (x_1, m, k_v) and (x_2, m, k_v) . Sample a random encrypted transcript $\hat{T}'_1$ and $\hat{T}'_2$ of the appropriate length for circuits $\hat{C}_1$ and $\hat{C}_2$. Invoke the Ligerio simulator on input statement $(\hat{C}'_1, (x_1, m, k_v), \hat{T}'_1)$, which produces an output com_1, π_1 , and then do the same for the second circuit. Finally, the simulator outputs a transcript in the properly interleaved order using these components. The argument that this simulator produces an indistinguishable output from the real invocation of the protocol relies on a reduction to the indistinguishability of the Ligerio simulator. As this is a bounded-size composition of two runs, this reduction is straightforward, and proceeds by hybrid argument. The original experiment consists of the real execution of the protocol. In the first hybrid step, the simulator for the $\hat{C}_1$ is invoked using the same k_v and correct mac value m on w to produce a transcript. These messages replace the real execution. A distinguisher here immediately breaks the ZK property of Ligerio on the first instance (using the auxiliary input of a true witness for the second theorem and w). In the next hybrid, the same is done for $\hat{C}_2$. At this point, the experiment no longer uses w_1, w_2 . In the next hybrid, the mac value m is randomly sampled instead of computed using w . This step is statistically close by the hiding property of the MAC. At this point, no witness is used in the simulator algorithm; in fact, this last hybrid is equivalent to the experiment in the definition of zero-knowledge for this protocol, which establishes the result. $\square$

Note that when the extractor for Liger is straightline (e.g., in the random oracle model), then the composed extractor for Protocol 3.8 is also straightline, and thus composes efficiently when run with other protocols.

4 Circuit design

4.1 ECDSA Signature Verification

This section describes the construction of an arithmetic circuit that verifies the existence of an ECDSA signature under public key $Q = (pk_x, pk_y)$ for the hashed message $e = H(m)$. Following the Sec-1 [Cer09] specification, which is identical to the ANSI X9.62.2005 standard, an ECDSA signature verification proceeds as follows:

Algorithm 3 ECDSA-Verify($Q, H(m), r, s$) [Cer09]

- 1: Derive integer e from $H(m)$.
 - 2: Verify that $r, s \in [1, q - 1]$.
 - 3: Compute $u_1 = es^{-1} \bmod q$ and $u_2 = rs^{-1} \bmod q$.
 - 4: Compute $R = (r_x, r_y) = G \cdot u_1 + Q \cdot u_2$ and verify $R \neq id$.
 - 5: Verify that $r = (r_x \bmod q)$.
-

Several of these steps occur over the integers or in the field $\mathbb{F}_q$, whereas our checks must be performed in the field $\mathbb{F}_p$. Towards this goal, the verification circuit defined in Alg 4 succeeds on an input x only if an easily computable tuple $t(x)$ induces Alg. 3 to succeed. Furthermore, the circuit is almost perfectly complete, with only a handful of cases that do not pass our checks but do pass the official spec—this is acceptable in all applications that we consider, even when malicious signers are considered.

Algorithm 4 Verification Circuit $V(Q, H(m), r, s, r_y)$

- 1: Derive integer e from $H(m)$.
 - 2: Verify that $r, s \in [1, q - 1]$.
 - 3: Verify that $Q, R = (r, r_y) \in E$ and $R \neq id$.
 - 4: Verify $id = G \cdot e + Q \cdot r - R \cdot s$
-

The main difference in the two procedures is that the later avoids computations of inverses modulo q and performs checks to ensure that points are on the elliptic curve group.

Theorem 10. *If $V(Q, H(m), r, s, r_y)$ succeeds where $r, s \in [0, 2^{\lceil \log p \rceil} - 1]$ and $r_y \in \mathbb{F}_p$, then ECDSA($Q, H(m), r, s$) from Sec-1 [Cer09] succeeds.*

Verifying a multi-scalar multiplication The most expensive step to verify in Alg. 4 is step 4, which verifies that $G \cdot e + Q \cdot r - R \cdot s$ is equal to the identity element. By providing the intermediate values as witnesses, we produce a low-depth circuit that computes the left-hand side. Specifically, the computation consists of a standard double-and-add loop over the bits of the exponents. In the i^{th} iteration, the bits e_i, r_i, s_i are used to index a table that contains eight combinations of the G, Q , and R elements. The current accumulator is doubled and the result of the table lookup is added. As described, the input of the $i + 1^{\text{st}}$ iteration would depend on the output of the i^{th} iteration, resulting in a deep circuit. Instead, we provide the outputs of each iteration as witnesses, and compare the output of the basic double-and-add block to the input for the next block. To further reduce the depth, the 7 non-zero table values are given as witnesses and verified by the circuit. A full

accounting for the resulting circuit size for ECDSA verification is given in Table 1. The circuit compiler exploits shared terms and thus the total is slightly less than the sum.

Table 1: Circuit size and depth for ECDSA verification.

	DEPTH	QUADS	TERMS	INPUTS
Multi-exponentiation	7	20,561	39,598	1,038
Range check + rest	12	-	-	-
Total	12	24,477	49,646	1,038

4.2 SHA-256

The SHA-256 hash function iteratively applies a compression function to successive 64-byte blocks of the input message. A large n -bit message m is first divided into 64-byte blocks; the basic block function is applied to the i^{th} block to produce a compressed 32-byte output, which is then combined with the $i + 1^{\text{st}}$ block. The final message block must be at most 55 bytes, as it is padded with zeros and then augmented by an 8-byte encoding of n .

To verify a message of at most $64k - 9$ bytes, our circuit evaluates k SHA-256 basic block functions. A basic block of the compression function takes as input eight 32-bit state values, designated by letters $A-H$, 64 bytes of the message, and produces a new state value. The block uses the Ch , Maj , Σ_0 , and Σ_1 functions and addition modulo 2^{32} .

$$\begin{aligned}
 \text{Ch}(E, F, G) &= (E \wedge F) \oplus (E \wedge \neg G) \\
 \text{Maj}(A, B, C) &= (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C) \\
 \Sigma_0(A) &= (A \ggg 2) \oplus (A \ggg 13) \oplus (A \ggg 22) \\
 \Sigma_1(E) &= (E \ggg 6) \oplus (E \ggg 11) \oplus (E \ggg 25)
 \end{aligned}$$

The state values B, C, D, F, G, H are assigned the values of A, B, C, E, F, G respectively. The values A and E are updated as

$$\begin{aligned}
 A &= W_i \boxplus K_i \boxplus H \boxplus \text{Ch}(E, F, G) \boxplus \Sigma_1(E) \boxplus \text{Maj}(A, B, C) \boxplus \Sigma_0(A) \\
 E &= W_i \boxplus K_i \boxplus H \boxplus \text{Ch}(E, F, G) \boxplus \Sigma_1(E) \boxplus \text{Maj}(A, B, C) \boxplus D
 \end{aligned}$$

where $\boxplus$ represents addition modulo 2^{32} .

Verifying Modular Addition Verification of addition modulo 2^{32} can be an expensive operation. The straightforward method uses a Boolean adder circuit, e.g. ripple-carry or parallel-prefix, to recompute the output. In fact, verification of the addition $A \boxplus B = C$ can be performed by a constant-depth circuit if A, B and C are provided as inputs by using the fact that the i -th carry bit can be inferred from the equation $C[i] = A[i] \oplus B[i] \oplus \text{carry}[i]$. Thus one only needs to verify that the carry has been propagated correctly, as opposed to actually propagating the carry. Going further, we present two more efficient methods to verify this operation that exploit properties of the finite field and the fact that we compute sums modulo 2^{32} of a small number t of terms $T[i]$, where concretely $t \leq 7$.

If the characteristic of the field is greater than $2^{32} \cdot t$, then the addition of t terms in the field does not overflow. Let $s = \sum_{0 \leq i < t} T[i]$ in the field arithmetic, and let r be the claimed value of the sum modulo 2^{32} , interpreted as a field element. Let $z = s - r$ in the field. Then $z = 0 \pmod{2^{32}}$ iff $\prod_{0 \leq i < t} (z - 2^{32}i) = 0$, where the latter check occurs by means of a tree of multiplications in the field.

Table 2: Circuit size and depth for 1 SHA-256 block over 2 different fields.

	PACKING	DEPTH	QUADS	TERMS	INPUTS
$\mathbb{F}_{P256}$	-	7	51,334	187,316	6,657
	2	9	66,760	217,040	3,585
	3	10	77,357	241,455	2,625
$\mathbb{F}_{2^{128}}$	-	13	66,747	107,610	6,657
	2	14	66,751	152,527	3,585
	3	15	74,842	168,030	2,625

If the characteristic of the field is small, as with $\text{GF}(2^{128})$, we apply the previous method to the multiplicative group of the field instead of the additive group. Let g be a generator of the multiplicative group, and assume that $\text{order}(g) > 2^{32}t$. Then $g^s = \prod_{0 \leq i < t} g^{T[i]}$ assumes one of t possible values $g^{r+2^{32}i}$ for $0 \leq i < t$, which can be compared against via a tree of multiplications in the field, as before.⁶

Table 2 reports the sizes of instantiations of our SHA256 circuits in both fields using different *packing* parameters. To reduce the number of inputs, and thus the size of the commitment, we can pack several *bit* witnesses into a single witness, and then apply a simple polynomial to extract the individual bits from the witness.

4.3 CBOR parsing

MDOC credentials are encoded in the Concise Binary Object Representation [BH20] format (CBOR). For the purposes of this paper⁷, a CBOR datum is either an integer (signed or unsigned), a boolean, an array of bytes, an array of CBOR datums, or a map from datum to datum. The first byte of the encoding of a datum consists of a three-bit *tag*, denoting the various types (integer, array, map, etc.), and a five-bit *count*. Small integers are encoded inline in the count. For larger integers, the count encodes the size of the integer, which is then stored in subsequent bytes of the document. If the tag indicates that the datum is an array or a map, the count similarly encodes the number of elements in the array or the number of pairs in the map, with the actual contents stored in subsequent bytes in the document.

Let $in[i]$ be the i -th byte of input, where $in[i]$ is itself an array of eight input wires each carrying a bit. In the context of our proof system, the parsing circuit is expected to assert the truth of statements such as the following:

The top-level document is a map that contains a pair whose key is the string `valueDigests` and whose value is another map. The `valueDigests` map contains a pair whose key is the string `org.iso.18013.5.1` and whose value is another map. The `org.iso.18013.5.1` map contains a pair whose key is the unsigned number 4 and whose value is a byte array of length 32, which a SHA256 hash of ...

To verify such assertions, our circuit is given witnesses that mark the positions of the relevant datums in the input; this enables the circuit to efficiently reconstruct the tree structure of the document. We employ a parallel-prefix algorithm to reduce circuit depth. In the “combinational” arithmetic circuit model, the whole input is given to a fixed-size circuit, and thus the size of documents are limited to N bytes for some constant N known

⁶In the special case where the field has a (multiplicative) root of unity of order 2^{32} , we can choose g to be such root, and obtain modulo- 2^{32} arithmetic “for free”. This special case does not apply to the fields used by our circuits.

⁷CBOR supports a few more cases such as floating-point numbers, which we ignore.

Algorithm 5 Sequential algorithm for tokenization.

```

1: procedure TOKENIZE( $N, in[]$ )
2:    $\triangleright$  Procedure DECODE (not shown) produces an array length where  $length[i]$  is the length
      in bytes of the token that starts at position  $in[i]$ .  $\triangleleft$ 
3:    $(token, length) \leftarrow \text{DECODE}(N, in)$ 
4:    $\triangleright$  Now set  $header[i] \leftarrow \text{true}$  if and only if a token actually starts a position  $i$ .  $\triangleleft$ 
5:    $header[0] \leftarrow \text{true}$ 
6:    $slen[0] \leftarrow length[0]$ 
7:   for  $i = 1$  to  $N - 1$  do
8:      $header[i] \leftarrow ((slen[i - 1] - 1) = 0)$ 
9:     if  $header[i]$  then
10:       $slen[i] \leftarrow length[i]$ 
11:     else
12:       $slen[i] \leftarrow (slen[i - 1] - 1)$ 
13:   return  $(token, header)$ 

```

at circuit-generation time. Similarly, because the model does not contain a stack to support arbitrarily nested arrays and maps, we restrict the parser to L nesting levels, for some constant L .

Toy CBOR To illustrate the main problems and solutions in CBOR parsing, we focus our discussion on parsing a context-free CBOR-like toy language defined below.

A *toy-CBOR* document consists of a sequence of *bytes* $in[i]$ that are grouped into *tokens*. The token starting at position i is defined as follows.

Case $0x00 \leq in[i] < 0x10$. The token is $\text{Int}(in[i])$, that is, a four-bit integer. The next token starts at position $i + 1$.

Case $in[i] = 0x10$. The token is $\text{Int}(in[i + 1])$, that is, the eight-bit integer stored at $in[i + 1]$. The next token starts at position $i + 2$.

Case $0x20 \leq in[i] < 0x30$. The token is $\text{Array}(in[i])$. The next token starts at position $i + 1$.

Case $in[i] = 0x30$. The token is $\text{Array}(in[i + 1])$. The next token starts at position $i + 2$.

Otherwise: Error.

In addition to the lexical structure of tokens, toy CBOR has a context-free syntactic structure which we informally define by giving a recursive-descent parser for it. Specifically, the parser reads the first token from the stream, consuming it. If the token is $\text{Int}(k)$, the parser returns the integer k . Otherwise the token is $\text{Array}(k)$, in which case the parser calls itself recursively k times, returning an array of length k containing the results of the recursive calls.

Full CBOR has both a richer token structure, including variable-length integers and strings, and a richer syntactic structure, including both arrays and maps, but the main difficulties already arise in our Toy version.

Lexer The first step is to identify token boundaries. Given input in of length N , Algorithm 5 produces an array $header$ such that $header[i]$ is true if and only if a token starts at position i . Thus, assertions of the form “an array starts at position i ” must assert $header[i]$ first.

Algorithm 6 Algorithm for tokenization with witnesses

```

1: procedure TOKENIZEW( $N, in[], header[]$ )
2:    $(token, length) \leftarrow \text{DECODE}(N, in)$ 
3:    $slen[0] \leftarrow length[0]$ 
4:   for  $i = 1$  to  $N - 1$  do
5:     if  $header[i]$  then
6:        $slen[i] \leftarrow length[i]$ 
7:     else
8:        $slen[i] \leftarrow (slen[i - 1] - 1)$ 
9:   assert( $header[0]$ )
10:  for  $i = 1$  to  $N - 1$  do
11:     $\text{assert}(header[i] \iff (slen[i - 1] - 1) = 0)$ 
12:  return ( $token, header$ )

```

We omit procedure `DECODE` in Algorithm 5 because it is complex but not insightful. One should think of it as combinational logic which, for each input position i , decodes the bits of $in[i]$ into a set of signals that indicate whether the token is an integer or an array and computes a field value $length[i]$ that represents the length of the token so that the next token starts at position $i + length[i]$. For toy CBOR, $length[i] \in \{1, 2\}$ can be computed solely from $in[i]$, whereas the real CBOR requires peeking at $in[i + 1]$ and possibly subsequent input bytes. Being actually a circuit and not a procedure, `DECODE` speculatively computes $length[i]$ for all positions i , irrespective of whether or not a token actually starts at position i . In the actual implementation, `DECODE` must also cope with invalid tokens and output a validity status, and we must assert that $header[i]$ implies that the token at position i is valid.

Reducing the depth of lexing Given that an upper bound to N is known at circuit-generation time, one could unroll the loop in Algorithm 5 and generate a circuit. However, such a circuit would be of depth $O(N)$, which is inefficient for a sumcheck-based prover. We now use well-known parallel-prefix techniques to reduce the depth of the circuit.

If $header[i]$ were always false in line 9 of Algorithm 5 (which is not the case), then the update equation for $slen$ would be $slen[i] = slen[i - 1] + A[i]$ for $A[i] = -1$, which is an instance of a *prefix sum* or *scan*. Because $slen[i] = \sum_{j \leq i} A[j]$, each $slen[i]$ can be computed in depth $O(\log n)$ simply by building an independent tree of adders for each i . Various algorithms, collectively called *parallel-prefix*, allow computation in depth $O(\log n)$ of $slen[i]$ for all i while maximizing reuse of common subexpressions. See Blelloch [Ble90] and references therein for details.

The full update equation

$$slen[i] = \text{if } header[i] \text{ then } length[i] \text{ else } slen[i - 1] + A[i], \quad (9)$$

again for $A[i] = -1$, has the form of a *segmented scan*. Logarithmic-depth circuits exist that can compute $slen[\cdot]$ given $header[\cdot]$, $length[\cdot]$, and $A[\cdot] = -1$ as inputs. See [Ble90, Section 1.5] for a general technique to transform segmented scans into unsegmented scans.

The segmented scan technique would solve our circuit-depth problem, except that segmented scans require the segment marker $header[\cdot]$ to be provided as input to the circuit, whereas in our case, $header[i]$ is computed as a function of $slen[i - 1]$ in line 8. To work around this problem, we provide booleans $header[i]$ as witnesses, one per byte input position, and use Algorithm 6 as our starting point instead of Algorithm 5.

It is not immediately obvious that Algorithm 6 and Algorithm 5 are equivalent, because Algorithm 6 computes $slen[\cdot]$ from an untrusted witness $header[\cdot]$ and then it asserts

Algorithm 7 Sklansky [Skl60] parallel prefix algorithm

```

procedure SCAN-ADD( $i_0, i_1, B[]$ )
  ▷ In-place, set  $B_{out}[i_0] = B_{in}[i_0]$  and  $B_{out}[i] = B_{in}[i] + B_{out}[i - 1]$  for  $i_0 < i < i_1$ .      ◁
  if  $i_1 - i_0 > 1$  then
     $i_m \leftarrow i_0 + \lfloor (i_1 - i_0) / 2 \rfloor$ 
    SCAN-ADD( $i_0, i_m, B$ )
    SCAN-ADD( $i_m, i_1, B$ )
    for all  $i : i_m \leq i < i_1$  do
       $B[i] \leftarrow B[i] + B[i_m - 1]$ 

```

Algorithm 8 Sequential algorithm for updating tree addresses.

```

procedure UPDATE-COUNTERS( $i, counters[], token[], header[]$ )
  ▷ Given the tree address counters[i] and the output (token, header) of the tokenizer, compute counters[i + 1].      ◁
   $counters[i + 1] \leftarrow counters[i]$ 
  if  $header[i]$  then
    Let  $l[i] \in \{l \in \text{Nat} : counters[i][l] \neq 0 \wedge \forall m > l : counters[i][m] = 0\}$ 
     $counters[i + 1][l[i]] \leftarrow counters[i][l[i]] - 1$ 
    if  $token[i] = \text{Array}(k)$  then
       $counters[i + 1][l[i] + 1] \leftarrow k$ 

```

that $header[.]$ is correctly computed from $slen[.]$, whereas Algorithm 5 interleaves the computation of $slen[.]$ and $header[.]$. The correctness of Algorithm 6 can be proven by induction on the loop variable i , in essence mimicking the control flow of Algorithm 5. If the assertion in line 9 holds, then $slen[0]$ and $header[0]$ are both correct. If the state variables are correct for $(i - 1)$ and the assertion in line 11 holds, then $header[i]$ is correct, and thus $slen[i]$ is correct as well because the update of $slen[i]$ is the same for both algorithms.

Of the many concrete parallel-prefix algorithms, we use the Sklansky [Skl60] form (see Algorithm 7), which has $O(n \log n)$ gates and depth $\lg n$. The Brent-Kung form [BK82], among others, requires only $O(n)$ gates, but it has twice the depth as the Sklansky algorithm. In the layered circuits required by sumcheck, values need to be propagated across each layer, and thus a lower depth is preferable even at the cost of extra gates.

Reconstruction of tree structure A toy-CBOR document ultimately represents a tree. One can view each token in the document as a tree node: A token $\text{Int}(k)$ represents a leaf, and a token $\text{Array}(k)$ represents an inner node with k children. On this tree, we define the *tree address* of a node as follows. The root has depth $l = 0$, and its tree address is the array $[1]$. The tree address of a node at depth $(l + 1)$ is $[p[0], p[1], \dots, (p[l] - 1), j]$, where p is the tree address of the parent, and j is child number. We use peculiar conventions that facilitate computation of tree addresses in a circuit. First, note the use of $(p[l] - 1)$ in the tree address of the child, as opposed to the perhaps more natural $p[l]$. Second, we number children in decreasing order left to right. The right-most child (the one parsed last) has $j = 1$, and the child number of a left sibling is one plus the child number of its right sibling. With these conventions, the last element in the tree address is always nonzero.

While the tree address is a variable-length array, in a circuit we represent it as an array of wires of fixed size L , where L is the maximum nesting depth supported by the circuit, implicitly padded with zeroes. Thus, in the actual circuit, all tree addresses are stored in an array $counters[i][L]$ of wires, indexed by position and nesting depth.

Assuming we have the tree address of every position i that starts a token (i.e., where

Table 3: Circuit size and depth for CBOR parsing over $\mathbb{F}_{2^{128}}$.

	MSG (B)	DEPTH	QUADS	TERMS	INPUTS
$\mathbb{F}_{2^{128}}$	247	37	210,785	261,236	2,227
	503	40	477,197	581,910	4,531
	1079	45	1,166,296	1,396,955	9,715
	1591	46	1,810,432	2,160,157	14,323
	2231	48	2,638,206	3,126,391	20,083
	2551	48	3,018,335	3,583,464	22,963

$header[i]$ is true), we can make assertions such as “this node at depth $(l + 1)$ is a child of that node at depth l ”, by comparing prefixes of the tree addresses up to l . Therefore, having the tree address available allows us to generate circuits that check assertions about the syntactic structure of the document.

The only remaining problem is now to design a circuit that computes the tree address. While tree addresses are defined for *tokens*, or equivalently, for positions i such that $header[i]$ holds, in a circuit it is more convenient to compute $counters[i]$ for every i . If $header[i]$ is false, we define $counters[i] = counters[i - 1]$, which is well-defined as long as $header[0]$ is true.

As before, we first present a sequential algorithm for computing tree addresses, and then convert it into a low-depth circuit. Our starting point is Algorithm 8, which computes $counters[i + 1]$ given $counters[i]$ and the output of the tokenizer. If $header[i]$ is false, the procedure sets $counters[i] \leftarrow counters[i - 1]$, as we postulated. Otherwise, let l (explicitly called $l[i]$ in the code) be the highest depth such that $counters[i][l] \neq 0$, which is the depth of the tree node by definition of tree address. Thus, $counters[i + 1][l]$ gets the tree address $counters[i][l]$ of its left sibling minus one, as per our conventions on the numbering of children. If the token at position i starts an array of length k , we extend the tree address by one depth $counters[i + 1][l + 1] \leftarrow k$, where, again by convention, k is the child number of the leftmost child in the array (the first array element).

As in the case of the tokenizer, one could unroll the circuit for all i , obtaining a deep circuit that computes all $counters[i]$ starting with $counters[0] = [1, 0, 0, \dots]$. To reduce the depth, we provide $l[i]$ as witness to break the dependencies, at which point the computation of $counters[i][l]$ is reduced to L segmented scan operations, one per depth l , plus assertions that the witnesses are correct.

Circuit sizes Table 3 presents the cost of parsing CBOR documents of varying sizes. The message sizes correspond to messages that can be hashed with SHA256 using an integral number of blocks. The table refers to one point in design space, but other tradeoffs are possible.

5 Evaluation

We implemented our ZK protocol in roughly 20,000 lines of C++ code with an additional 15,600 lines corresponding to unit tests or benchmarks. Our implementation uses only a single thread; our reported wall-clock timings thus reflect total computation.

We report benchmarks for several components of an anonymous credential system. We collected these performance numbers using the C++ benchmark package on an amd_64 machine, a cloud x86_64 machine, a Pixel 9 phone, an iPhone 15 plus, and an Apple M4 Macbook Air. The amd_64 platform is an AMD Ryzen 9 7950X released in September 2022. This machine has Zen4 cores with base clock 4.5 Ghz; we only used 1 thread for

Table 4: NTT benchmarks over different fields.

		Time (in ms)					
		2^{12}	2^{14}	2^{16}	2^{18}	2^{20}	2^{22}
amd64	$\mathbb{F}_{2^{128}}$	0.061	0.281	1.28	5.75	25.7	137
	$\mathbb{F}_{p^{64}}$	0.046	0.235	1.19	5.48	23.6	105
	$\mathbb{F}_{p^{128}}$	0.126	0.609	3.02	13.4	59.2	276
	$\mathbb{F}_{p^{256^2}}$	0.433	2.03	9.51	43.6	223	1160
cloud	$\mathbb{F}_{2^{128}}$	0.042	0.194	0.883	4.14	18.4	88.8
	$\mathbb{F}_{p^{64}}$	0.064	0.338	1.79	8.22	35.4	152
	$\mathbb{F}_{p^{128}}$	0.177	0.860	4.35	19.4	85.1	378
	$\mathbb{F}_{p^{256^2}}$	0.658	3.08	14.3	73.1	362	1800
Pixel 9	$\mathbb{F}_{2^{128}}$	0.061	0.305	2.38	10.3	47.7	196
	$\mathbb{F}_{p^{64}}$	0.066	0.309	1.71	7.32	33.4	165
	$\mathbb{F}_{p^{128}}$	0.196	0.972	4.85	21.3	96.3	452
	$\mathbb{F}_{p^{256^2}}$	0.651	3.02	14.1	67.0	322	1410
iPhone 15+	$\mathbb{F}_{2^{128}}$	0.034	0.163	0.744	3.40	15.5	77.2
	$\mathbb{F}_{p^{64}}$	0.057	0.268	1.44	6.24	29.3	128
	$\mathbb{F}_{p^{128}}$	0.155	0.767	3.70	17.1	75.1	335
	$\mathbb{F}_{p^{256^2}}$	0.454	2.18	10.0	45.5	219	1106
M4 Air	$\mathbb{F}_{2^{128}}$	0.024	0.114	0.513	2.66	12.0	52.7
	$\mathbb{F}_{p^{64}}$	0.034	0.160	0.913	3.91	18.3	90.1
	$\mathbb{F}_{p^{128}}$	0.103	0.508	2.57	11.5	52.6	228
	$\mathbb{F}_{p^{256^2}}$	0.338	1.67	7.70	37.6	181	954

benchmarking and allowed the machine to scale its own clock. The “cloud” platform is a c4-highcpu-8 instance, running in the central1-c availability zone in Google Cloud. This machine has four Intel Xeon Platinum 8581C CPU (cpu family 6, model 207) cores running at approximately 3.2GHz with 16gb of RAM. The Pixel 9 platform, released August 2024, contains a Google Tensor G4 whose performance core is a Cortex-X4 (3.1 GHz). The iPhone 15+, released September 2023, uses an A16 Bionic chip with 2 performance cores (3.46 GHz). The Mac M4 Air, released March 2025, contains 4 performance cores that claim to operate at approximately 4.4 Ghz. When collecting timings on phones, benchmarks are run on the performance core and spaced to reduce thermal throttling.

We use SHA256 to implement the collision-resistant hash function, and the random oracle, resulting in a 128-bit computational security parameter. For our Ligerio commitment, we use a rate 1/7 code and open 132 columns, which corresponds to roughly 110 bits of security. We store prime-field elements in Montgomery form and implement field multiplication using the 64x64-bit scalar multiplier instead of SIMD instructions. For $\text{GF}(2^{128})$ we use 128-bit SIMD registers on both arm64 and x86_64, and implement multiplication using the 64x64-bit carryless multiplier, which is implemented by x86_64 and by the arm64 AES instruction set. We do not use AVX256, AVX512, or GFNI.

5.1 Algebraic primitives

The first benchmark in Table 4 shows the performance of our NTT implementation across different fields and processors. The $\mathbb{F}_{p^{128}}$ and $\mathbb{F}_{p^{64}}$ fields both have at least 2^{32} roots of unity. As noted before, $\mathbb{F}_{p^{256}}$ does not have roots of unity, and therefore we must

Table 5: Reed-Solomon encoding performance.

		Time (in ms)					
		2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
amd64	$\mathbb{F}_{2^{128}}$	0.052	0.247	1.14	5.50	24.3	114
	$\mathbb{F}_{p64}$	0.104	0.538	2.73	12.3	53.5	258
	$\mathbb{F}_{p128}$	0.309	1.46	6.90	30.4	135	659
	$\mathbb{F}_{p256}$	1.01	4.67	21.3	97.0	503	2570
cloud	$\mathbb{F}_{2^{128}}$	0.037	0.173	0.797	3.75	17.5	79.7
	$\mathbb{F}_{p64}$	0.142	0.727	3.75	17.3	74.7	333
	$\mathbb{F}_{p128}$	0.450	2.12	10.1	44.8	200	900
	$\mathbb{F}_{p256}$	1.59	7.37	33.8	168	802	3940
Pixel 9	$\mathbb{F}_{2^{128}}$	0.050	0.242	1.26	5.87	29.2	145
	$\mathbb{F}_{p64}$	0.148	0.743	3.98	17.2	82.1	381
	$\mathbb{F}_{p128}$	0.476	2.51	12.0	53.5	239	1089
	$\mathbb{F}_{p256}$	1.62	7.41	33.9	159	743	3204
iPhone 15+	$\mathbb{F}_{2^{128}}$	0.032	0.154	0.726	3.29	15.3	73.2
	$\mathbb{F}_{p64}$	0.131	0.574	2.87	13.3	61.2	324
	$\mathbb{F}_{p128}$	0.384	1.76	8.10	36.4	166	916
	$\mathbb{F}_{p256}$	1.15	5.31	23.9	107	497	2420
M4 Air	$\mathbb{F}_{2^{128}}$	0.022	0.111	0.503	2.23	11.2	60.5
	$\mathbb{F}_{p64}$	0.076	0.346	1.94	8.29	42.3	187
	$\mathbb{F}_{p128}$	0.247	1.23	5.78	25.8	120	522
	$\mathbb{F}_{p256}$	0.781	3.66	16.7	76.7	361	1982

use a quadratic extension, and the specialized *Real FFT* algorithm to introduce them for computing the NTT. As a result, we see roughly 4-5x penalty in performance in this field versus the 128-bit prime fields. In all cases, the $\text{GF}(2^{128})$ field is the most performant field by a factor of at least 2x. The $\mathbb{F}_{p64}$, while not sound, is presented as a baseline to gauge our implementation against others and highlight the overheads between different fields and extensions.

Reed-Solomon Encoding Table 5 reports on Reed-Solomon encoding benchmarks used in the Liger commitment. Each test measures the time to encode one array of size n into a codeword of size $4n$ (i.e., rate $1/4$). Recall from §3 that our Reed-Solomon encoding, or polynomial interpolation algorithm for prime fields relies on a convolution, which itself requires 2 NTTs and multiplication by pre-computed constants. In contrast, the encoding in $\text{GF}(2^{128})$ field uses 1 bidirectional FFT and 4 FFTs of the same size as the input. The table confirms that this results in 2–3x speedups.

Comparison to Linear-time codes Golovnev *et. al* [GLS⁺23] and Xie, Zhang, and Song [XZS22] propose the use of linear-time encodable error-correcting codes to construct a polynomial commitment scheme. It is not immediately clear how to augment either to support ZK verification of linear and quadratic constraints (instead, both of those schemes were designed to answer the final query in a sumcheck-based verifiable computation protocol). Moreover, such codes have poor distance properties, and thus require many queries to the oracle, resulting in proof sizes that are an order of magnitude larger. For example, the Brakedown scheme is described with distance $1/20$ and rate $3/5$,

Table 6: Encoding time for Linear-time encodable codes over a 128-bit field.

		Time (in ms)				Queries needed
		2^{14}	2^{16}	2^{18}	2^{20}	
Brakedown	cloud	4.95	19.5	79.1	364	6,593
	Pixel 9	5.27	20.9	79.7	354	6,593

Table 7: Timing for our ZK proof of knowledge of an n -block pre-image to the SHA256 circuit (§4.2) over $\text{GF}(2^{128})$. The “Total Prover ZK” row reports the total prover time (including verify and commitment, etc) to produce the proof.

		OurZK Protocol, Time (ms)					
		$n = 1$	2	4	8	16	32
amd64	IP Step	6.13	12.0	23.8	47.9	96.1	192
	Total Prover ZK	12.5	20.2	35.6	67.4	130	243
cloud	IP Step	5.4	10.6	21.1	42.4	85.6	182
	Total Prover ZK	13.5	20.9	36.3	66.7	133.5	244
Pixel 9	IP Step	8.18	16.7	35.5	75.2	153	316
	Total Prover ZK	22.0	34.5	61.9	116	223	426
iPhone 15+	IP Step	4.66	9.08	18.1	36.5	74.2	154
	Total Prover ZK	12.8	20.5	35.7	65.6	117	230
M4 Air	IP Step	3.03	5.60	12.1	23.1	44.7	99.2
	Total Prover ZK	5.51	9.00	16.0	30.4	58.9	116

and therefore requires at least 6500 queries.

Table 6 also shows that known linear-time codes are *slightly slower* than our $\text{RS } \mathbb{F}_{p_{64}}$ code and substantially slower than our $\text{GF}(2^{128})$ code for the relevant parameter regime. We benchmark the Brakedown implementation of linear-time encodable codes over 128-bit fields. Because that scheme requires at least 6,500 queries to the codeword, their benchmark begins at 2^{14} . For our parameter regimes, the coefficient for their linear-time encoding scheme seems to be larger than our quasi-linear Reed-Solomon encoding in $\text{GF}(2^{128})$, as our implementation begins at +4x faster and remains 3x faster at $n = 2^{20}$.

5.2 SHA256 hash verification

Many ZK proof systems publish benchmarks for proving knowledge of a SHA256 pre-image. Our $\text{SHA256}_n(e, m)$ circuit verifies that the private message m can be hashed using at most n SHA256 block operations (i.e., $|m| < 64n - 9$ bytes) and hashes to the public value e . This circuit corresponds to the one described in §4.2 with bit-packing 2. Our timing results for 1–32 blocks appear in Table 7. The “IP Step” row reports the time required to create the verify transcript for the circuit. The Total ZK row reports the total prover time (including verify and commitment, etc) to produce the proof. Table 8 reports commitment parameters required for one block of SHA256.

5.2.1 Comparison with Ligero

As mentioned in the introduction, Ligero requires committing to a 3 witnessess for every quadratic constraint in the circuit. As a result, the size of the Ligero commitment for a

Table 8: Commitment size parameters for 1 block of our SHA256-preimage proof in different fields.

SHA256 ₁	$ w $	Witness/row	Row size	# rows
$\mathbb{F}_p$	3,100	682	2^{12}	12
$\text{GF}(2^{128})$	2,810	341	2^{11}	20

Table 9: Size of the Ligero instance for n blocks of SHA256 as derived from an optimal R1CS expression of SHA256.

		SHA256 _n ($\cdot$)		
		$n = 1$	2	4
R1CS	Witnesses	29,853	60,381	121,437
	Constraints	29,725	60,053	120,709
Ligero	Total Witnesses	129,500	261,484	525,452
	Quadratic	29,725	60,053	120,709
	Witness/row for w	3,149	6,426	6,426
	Quadratic rows	1,511	3,149	3,149
	Encoded Row size	2^{15}	2^{16}	2^{16}
	Number of rows	104	103	201

given circuit C is $O(|C|)$, which can be many times larger than the corresponding verify transcript for $C(x) = 0$. In this section, we provide concrete evidence for this claim with respect to the SHA256 circuit.

Because Ligero performs better with a circuit that optimizes for the total number of quadratic constraints, we used the `circom` system to generate an R1CS instance for our family of SHA256_n circuits described above. Recall that an R1CS instance consists of a witness tableau w and a list of constraints in the form of $A \cdot B - C = 0$ where A, B, C are linear combinations of a witness vector w . Smaller R1CS instances naturally require fewer multiplications to verify. We converted this R1CS into Ligero constraints following the description in their paper. Namely, for each A, B, C , we introduce new Ligero witnesses $\alpha_i, \beta_i, \kappa_i$ respectively that are constrained to be equal to the linear combination of the original witness w . We then introduce a new witness value $\gamma_i = \alpha_i \cdot \beta_i$, and then add a linear constraint that verifies $\gamma_i - \kappa_i = 0$. In total, we introduce 4 new witnesses, 4 linear constraints and 1 quadratic constraint for each R1CS constraint. For R1CS constraints that have an empty C term, i.e., $C = 0$, we apply a simple optimization to remove the extra witness value and the fourth linear constraint. Altogether, we form a Ligero commitment using all of the base R1CS witnesses plus the extra witnesses needed by Ligero to check R1CS. Table 9 reports both of these figures. In addition, each quadratic constraint on a witness requires adding a *copy* of the two terms and their product as witnesses. All of these witnesses are partitioned into rows of size w , with the additional constraint that the copies of the three witnesses for a quadratic constraint need to be on separate rows at the same index, and can only include half as many per row. Achieving ZK also requires reserving spots in each row for a random pad, and adding 2 additional random rows. A number of additional constraints govern the size of w . For each Ligero commitment, we loop over all feasible parameters, and chose a parameter set that minimizes the size of the commitment. These optimal choices are also presented in Table 9.

We benchmark our implementation of Ligero on these instances for c48 and report timings in Table 10. For these problems, the most expensive component of Ligero is the commitment stage. For soundness, Ligero should use a field of size at least 2^{128} , which

Table 10: Ligero ZK Prover time for verifying the SHA256 pre-image of an n -block message. The first line reports the time to Commit to the witness tableau. This benchmark was run using the $\mathbb{F}_{p64}$ field, which is not sufficient for soundness. Ligero is essentially 20x slower on c48.

		Time (ms)		
		$n = 1$	2	4
cloud	Commit	222	384	736
	Total ZK Prover	273	493	939
	Overhead wrt this work	20x	23.5x	26x
[WHV24]		250	450	750

can perhaps be implemented using a degree-2 or degree-4 field extension of a smaller base field. To be fair, we report benchmarks using a 64-bit prime. While this parameter setting is not as sound as our own implementation, it demonstrates that our system is at least *roughly 20x faster* for $n = 1, 2$, or 4 block messages, thus supporting our introductory claim. Ultimately, our numbers in Table 10 roughly match those presented by Wang, Hazay, and Venkitasubramaniam in the optimized Ligetron [WHV24, Fig 6(a)] implementation, which shows roughly 250ms, 450ms, 750ms for $n = 1, 2, 4$ blocks respectively; albeit the Ligetron benchmark uses 4 threads on a 4-core machine with a 50-bit prime, and higher statistical security parameter, so the comparison is imperfect.

The $\approx 20x$ performance improvement of our system over pure-Ligero can be partly explained by inspecting the witness size as per Table 8 for a single SHA256 block. The *full* witness in our proof system contains roughly 9x fewer rows (11 vs 104) with a block size that is 4x smaller, roughly reflecting a 36x reduction in work to produce the commitment; however, our results use a 128-bit field and thus incur a roughly 2x overhead with respect to $\mathbb{F}_{p64}$ used in Table 10.

5.2.2 Other Comparisons

Diamond and Posen [DP25, DP24] implement a commitment scheme and a verifiable computation protocol (not zero-knowledge) in the Binius [bin] system. They provide a SHA256 benchmark that verifies 32 independent applications of the SHA compression function; their x64 implementation also exploits native `gfni` instructions. On cloud, their benchmark, when run on a single core, runs in 498ms (versus our verify which runs in 192ms). Although their benchmark numbers are larger than ours, the performance gap closes as n increases; their system incorporates ideas that may also apply and speedup our implementation in future work.

The stone-prover is an implementation of a succinct argument system based on the STARK protocol [BSBHR18] that requires no trusted setup parameters. Unlike our system which handles arithmetic circuits, this system handles general *programs* that are expressed for a bespoke *CPU*. This generality imposes an inherent overhead; prior published benchmarks [GLS⁺23] show that the STARK prover time is larger than Ligero. To confirm, on cloud, producing a proof that verifies SHA256 on a 1 block message required a *degree bound* of 4,194,304 and took 106,000ms of prover time. Processing a 2-block message took the same time (as it also fit into the degree bound). The 4-block message could not be processed on our cloud machine due to memory limits.

We did not benchmark SHA256 on proof systems which require a trusted parameter. Prior work [GLS⁺23] already shows that these systems require more prover time.

Table 11: Benchmark for verification of n ECDSA signatures. The circuit used here is described in §4.1 over field $\mathbb{F}_{p_{256}}$. The Total Prover time consists of the Ligero commitment time, the time to produce the Verify transcript, and other small steps.

		Time (ms)		
		$n = 1$	2	3
amd64	Ligero com	18.9	22.3	26.9
	Verify transcript	7.77	15.6	21.9
	Total ZK	29.0	41.0	53.0
	Verifier	18.8	26.0	32.0
cloud	Ligero com	25.3	29.6	35.9
	Verify transcript	10.2	20.3	28.7
	Total ZK	38.4	54.1	69.6
	Verifier	24.9	34.8	42.9
Pixel 9	Ligero com	34.5	40.7	48.9
	Verify transcript	14.0	29.0	41.2
	Total ZK	53.3	75.7	98.1
	Verifier	33.5	46.5	57.2
iPhone 15+	Ligero com	18.6	22.1	27.2
	Verify transcript	8.0	16.7	24.0
	Total ZK	30.7	43.3	56.6
	Verifier	20.8	28.8	35.8
M4 Air	Ligero com	13.1	15.4	18.9
	Verify transcript	5.9	12.7	16.3
	Total ZK	20.4	29.8	37.9
	Verifier	14.3	21.2	23.8

5.3 ECDSA signature verification

We now report performance results on this problem: Given (e, pk) , prove knowledge of (r, s) such that $\text{ecdsaVerify}(pk, e, r, s) = 1$. A related version of the problem, required in the MDOC credential use-case, hides pk as a witness. As described in §4.1, our circuit for both of these problems uses 1085 witnesses to complete the verification in low-depth.

Table 11 reports the basic ZK benchmarks for this problem. This benchmark demonstrates that contrary to folklore, a zero knowledge proof of possession of a signature for ECDSA—which requires roughly 20ms to 50ms depending on the hardware—is within a small factor of performance for a basic BBS+ proof of possession of a signature.

Comparison to the BBS+ scheme We ran the `pairing_crypto` package [Pai24] with the BLS12-381 curve and SHA256 on signatures that included 100 attributes and presentations that revealed 3 attributes. On c48, the BBS+ proof generation step required 10.2 ms and the verify step required 8 ms; our ECDSA proof generation method is within $\approx 4\times$ the performance. However, the BBS microbenchmark does not account for additional tasks needed to perform a credential show operation, such as verification that the certificate has

not expired.

Comparisons to other ZK-ECDSA Woo et al. [WOS⁺25] present a ZK proof of possession of an ECDSA signature using a variant of the Spartan ZK system with an appropriate ECDSA finite field. Spartan is also based on a sumcheck primitive, but instead of using layered circuits, it processes R1CS (which requires more committed witnesses). As a result, their prover runs in 900ms, or at least 25X slower than ours. In addition, their optimization specifically relies on a “sidecar” protocol in which the public key pk is known to the verifier; this requirement does not work when pk must be hidden in the witness commitment, e.g., when verifying the device-binding for mdoc signatures in which the device public key must be private. As a result, this technique cannot be used for legacy credential applications like MDOC.

The `circom-ecdsa` project [0xP22] solves a similar task for `secp256k1` using the Groth16 proof system; it requires both a trusted setup string and bilinear pairings. The trusted parameter is 973MB, and proof generation required 140,000ms when restricted to a single core. The `spartan-ecdsa` [Per23] project claims to produce a ZK-proof, but it only hides the s component of the signature, while requiring R to be public—it is therefore not a full ZK proof of knowledge of a signature and does not work for anonymous credentials.

6 Applications

We apply our proof system to applications for digital identity.

6.1 A simple anonymous credential

Belenkiy et al. [BCKL08] introduce the notion of a P-signature as a component that yields a construction of an anonymous credential scheme. Briefly, a P-signature scheme lets a user (1) obtain a signature on a committed message without revealing the message, and (2) construct a non-interactive zero-knowledge proof of knowledge of $(F(msg), \sigma)$ such that $\text{VerifySig}(pk, msg, \sigma) = 1$. In the formal security notion, Belenkiy et al. [BCKL08, Definition 3] write, “Note that we require that only the user’s input msg is hidden from the issuer, but not necessarily the user’s output σ . The reason that this is sufficient is that in actual applications (for example, in anonymous credentials), a user would never show σ in the clear; instead, he would just prove that he knows σ .” In this way, the formal definition is weaker than that of blind signatures that are sometimes required for anonymous credentials. It suffices for the user to produce a public key (e.g., for ECDSA, as this is a commitment to the corresponding secret key for this scheme), and incorporate the public key as a component in the signature. In this way, this step is already subsumed in many modern identity protocols which require device binding. It is also clear that property (2), sometimes abbreviated as the *proof of possession*, for the P-signature is the difficult property to achieve in prior work. This proof of possession is the same primitive that we construct above in §4.1.

In analogy to [BCKL08, Theorem 1], we claim

Theorem 11 (Informal). *If ECDSA is a secure signature scheme, and Protocol 2.5 is a zero-knowledge argument scheme, then there exists a secure P-signature scheme for ECDSA.*

Finally, applying Lemmas 2 and 3 from [BCKL08] imply the existence of a formal anonymous credential scheme from ECDSA (and implicitly SHA-256).

We report details to the practitioner on how to instantiate the *proof of possession* required by a P-signature construction for a few concrete schemes.

Simple document format We designed a document format for credentials that is based on the machine readable zone (MRZ) format used in passports and then report on the construction of (2).

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
firstname																																
lastname																																
dob								G	age over x								$validFrom$								$validUntil$							
dpk_x																																
dpk_y																																
issuerid				Attributes																												

Figure 2: A simple format for a 183-byte credential.

In Fig 2, we describe a 183-byte credential that allows for at least 60 standard attributes to be encoded, in addition to a device public key for device-bounded checks, and time values to indicate the validity period. Notably, this format requires only 3 SHA256 blocks to hash, and it requires no parsing to determine the byte-location of an attribute. For this credential, we develop a simple benchmark to prove that the credential holder is older than 18 (which amounts to checking the value of byte 74 in the format). See Algorithm 9 for the specific theorem being proven.

Algorithm 9 Toy Credential Verification

Require: $x = (pk, a, id, tr, now)$ and $w = (MSO, pk_{dx}, pk_{dy})$

1: Verify the following constraints:

$$\begin{aligned}
 e_1 &= \text{SHA256}(\text{MSO}[0 : 183]) \\
 a &= \text{MSO}[id] \\
 (pk_{dx}, pk_{dy}) &= \text{MSO}[96 : 160] \\
 t_{start} &= \text{MSO}[48 : 56] \\
 t_{end} &= \text{MSO}[56 : 64] \\
 t_{start} &< t_{now} < t_{end} \\
 \text{true} &= p256.verify((r_1, s_1), e_1, PK_{II}) \\
 \text{true} &= p256.verify((r_2, s_2), H(tr || hdr), (pk_{dx}, pk_{dy}))
 \end{aligned}$$

The benchmark in Table 12 shows that this toy format achieves performance that is within a factor of BBS+ proof verification (without incorporating the time to format the BBS+ attributes or check credential validity).

6.2 Proof of possession for MDOC credentials

ISO standard 18013-5 [18021] specifies a digital identity format that is used in mobile driver licenses and national identity formats. For example, the Arizona, California, Colorado, Georgia, Maryland and New Mexico issue driver licenses in the MDOC format to certain mobile devices. See Appendix A for details.

The MDOC format supports a form of *selective disclosure* in which the holder of an MDOC can reveal a subset of the attributes that are asserted in the credential. This disclosure format is implemented by including salted hashes of the attribute values in

Table 12: Toy credential benchmark over $\mathbb{F}_{p^{256}}$.

	DEPTH	QUADS	TERMS	INPUTS
Toy Circuit	12	592,555	1,799,704	20,171

	Prover Time (ms)	Proof size (kb)
amd64	290	291
cloud	396	-
Pixel 9	572	-
iPhone 15+	289	-
M4 Air	223	-

the credential format that is signed by the issuer. The salted-hash technique has two well-known privacy flaws. If Alice uses the same MDOC credential to assert identities to relying parties P_1 and P_2 , it is possible for P_1 and P_2 to combine their views of the presentation protocol in order to track the user across sessions. For example, the expiration times, or device key values can be used to track a user or link a user between different relying parties. A simple counter-measure is to require the user to use *different* MDOCs for each relying party—this measure comes at a cost for both the issuer, who now must issue 1000s of mdocs to each user, and the user, who must maintain state to ensure the unique property. A more serious flaw enables the issuer of the MDOC to collaborate with the relying party in order to identity users across sessions. This flaw has delayed the deployment of MDOC for some applications.

ZK for MDOC A solution to the above issuer-RP collusion problem for the MDOC format is to modify the presentation protocol so that the user instead produces a zero-knowledge proof that “their mdoc verifies with respect to the requested attributes.” Specifically, we aim to convince the verifier of the theorem statement shown in Algorithm 10 while leaking no extra information:

Algorithm 10 MDOC Verification

Require: $x = (PK_{II}, attr, Z, tr, time)$ and w consists of a 2551 byte string MSO, a length len , a hash e_1 , a hash h_2 , an index X , a signature pair (r_1, s_1) , a parsing witnesses c , a 32-byte nonce, a pair (pk_{dx}, pk_{dy}) , a pair of strings t_{start}, t_{end} , a header hdr and a signature (r_2, s_2)

1: Verify the following relations:

$$\begin{aligned}
e_1 &= \text{SHA256}(\text{MSO}[0 : len]) \\
h_2 &= \text{MSO}[\text{valueDigests}][\text{org.iso.18013.5.1}][X] \\
h_2 &= \text{SHA256}(\text{nonce}, attr, Z) \\
(pk_{dx}, pk_{dy}) &= \text{MSO}[\text{deviceKeyInfo}][\text{deviceKey}][-2, -3] \\
t_{start} &= \text{MSO}[\text{validityInfo}][\text{validFrom}] \\
t_{end} &= \text{MSO}[\text{validityInfo}][\text{validUntil}] \\
t_{start} &< t_{now} < t_{end} \\
\text{true} &= p256.verify((r_1, s_1), e_1, PK_{II}) \\
\text{true} &= p256.verify((r_2, s_2), H(tr || hdr), (pk_{dx}, pk_{dy}))
\end{aligned}$$

Importantly, the public values for this theorem are $(PK_{II}, attr, Z, tr, time)$ —these values are shared between the prover (the user with the credential) and the verifier (the relying party). Here, PK_{II} is the public key of the identity-issuer, Z is an attribute value to disclose, e.g., like the “age_over_18” boolean, and $attr$ is the name of this attribute (as written in the MSO), tr is the liveness transcript, and t_{now} is used to verify that the mdoc has not expired. The rest of the values in the statement are hidden, in the sense that the verifier is convinced that such values exist, but does not learn them through the interaction. Notice that the private witness h_2 is compared twice: once as the a string included in the MSO and once as the result of the hash of a concatenation of private and public strings. Performance is reported in Table 13 on several processors; all figures are again measured on a *single-core* using a *single thread*. The most expensive part of the proof generation concerns the SHA256 hash of the MSO. Our benchmark implements the two-field optimization, with the SHA256 circuit verified in $GF(2^{128})$.

Table 13: Prover time for ZK MDOC presentation of the age-over-18 attribute for an MSO of at most 2231 bytes. The circuit corresponds to Alg. 10.

	Prover (ms)	Verifier (ms)
amd_64	526	252
cloud	589	286
Pixel 9	931	508
iPhone 15+	437	229
Mac M4 Air	294	142

Extensions This basic application can be conveniently extended to support more privacy features. For example, it is easy to also hide the identity of the issuer by proving that the issuer is a member of a certified list of issuers. It is possible to support the disclosure of multiple attributes instead of 1 at little cost. Finally, it is possible to support the revocation of MDOC credentials in zero-knowledge by using a ZK list-membership technique the authors have documented in specification forums.

References

- [0xP22] 0xParc. Circom-ecdsa. <https://github.com/0xPARC/circom-ecdsa>, 2022.
- [18021] ISO/IEC FDIS 18013-5. Personal identification—iso-compliant driving licence—part 5: Mobile driving licence (mdl) application, 2021.
- [AHIV23] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. Liger: Lightweight sublinear arguments without a trusted setup. *Des. Codes Cryptogr.*, 91(11):3379–3424, 2023. doi:10.1007/s10623-023-01222-8.
- [BCKL08] Mira Belenkiy, Melissa Chase, Markulf Kohlweiss, and Anna Lysyanskaya. P-signatures and noninteractive anonymous credentials. In Ran Canetti, editor, *TCC 2008: 5th Theory of Cryptography Conference*, volume 4948, 2008. doi:10.1007/978-3-540-78524-8_20.
- [BH20] Carsten Bormann and Paul E. Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, December 2020. URL: <https://www.rfc-editor.org/info/rfc8949>, doi:10.17487/RFC8949.

- [bin] Binius implementation. gitlab.com/IrreducibleOSS/binus.git.
- [BK82] Brent and Kung. A regular layout for parallel adders. *IEEE Transactions on Computers*, C-31(3):260–264, 1982. doi:10.1109/TC.1982.1675982.
- [Ble90] Guy E. Blelloch. Prefix sums and their applications. Technical Report CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, November 1990.
- [BOGG⁺88] Michael Ben-Or, Oded Goldreich, Shafi Goldwasser, Johan Håstad, Joe Kilian, Silvio Micali, and Phillip Rogaway. Everything provable is provable in zero-knowledge. In Shafi Goldwasser, editor, *Advances in Cryptology – CRYPTO’88*, volume 403, 1988. doi:10.5555/88314.88333.
- [Bra99] Stefan Brands. *Rethinking Public Key Infrastructure and Digital Certificates*. PhD thesis, Eindhoven Institute of Technology, 1999.
- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity, 2018.
- [CCH⁺18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-shamir from simpler assumptions, 2018.
- [CDL16] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous attestation using the strong diffie hellman assumption revisited. In Michael Franz and Panos Papadimitratos, editors, *Trust and Trustworthy Computing*, 2016. doi:10.1007/978-3-319-45572-3_1.
- [Cer09] Certicom. Sec 1: Elliptic curve cryptography, v2.0. Standards document, 2009.
- [Cha85] David Chaum. Security without identification: Transaction systems to make big brother obsolete. *Communications of the ACM*, 10(28):1030–1044, 1985. doi:10.1145/4372.4373.
- [CL01] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In Birgit Pfitzmann, editor, *Advances in Cryptology – EUROCRYPT 2001*, volume 2045, 2001. doi:10.1007/3-540-44987-6_7.
- [CMT12] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. Practical verified computation with streaming interactive proofs. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 90–112, 2012. doi:10.1145/2090236.2090245.
- [DLFKP16] Antoine Delignat-Lavaud, Cedric Fournet, Markulf Kohlweiss, and Bryan Parno. Cinderella: Turning shabby x.509 certificates into elegant anonymous credentials with the magic of verifiable computation. In *2016 IEEE Symposium on Security and Privacy*, San Jose, CA, USA, May 22–26, 2016. IEEE Computer Society Press. doi:10.1109/SP.2016.22.
- [DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. *Cryptology ePrint Archive*, Paper 2024/504, 2024. URL: <https://eprint.iacr.org/2024/504>.

- [DP25] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, 2025. URL: <https://eprint.iacr.org/2023/1784>, doi:10.1007/978-3-031-91134-7_4.
- [GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: interactive proofs for muggles. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, 2008. doi:10.1145/269943.
- [GLS⁺23] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic snarks for r1cs. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082, 2023. doi:10.1007/978-3-031-38545-2_7.
- [Gow92] Roderick Gow. Cauchy’s matrix, the Vandermonde matrix and polynomial interpolation. *Bulletin of the Irish Mathematical Society*, 28:45–52, March 1992.
- [Knu97] Donald E. Knuth. *The art of computer programming, volume 2 (3rd ed.): seminumerical algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 1997.
- [KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. How to prove false statements: Practical attacks on fiat-shamir. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005, 2025. doi:10.1007/978-3-032-01887-8_1.
- [KZM⁺15] Ahmed Kosba, Zhichao Zhao, Andrew Miller, Yi Qian, T-H. Hubert Chan, Charalampos Papamanthou, Rafael Pass, abhi shelat, and Elaine Shi. CØCØ: A framework for building composable zero-knowledge proofs. eprint/2015/1093, 2015.
- [LANHC16] Sian-Jheng Lin, Tareq Y. Al-Naffouri, Yung-Hsiang S. Han, and Wei-Ho Chung. Novel polynomial basis with fast Fourier transform and its application to reed-solomon erasure codes. *IEEE Transactions on Information Theory*, 62(11):6284–6299, 2016. doi:10.1109/TIT.2016.2608892.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yung-Hsiang S. Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 316–325. IEEE Computer Society Press, 2014. doi:10.1109/FOCS.2014.41.
- [LCK⁺18] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast Fourier transform. In *ISAAC’18*, 2018. URL: <https://arxiv.org/abs/1802.03932>, arXiv:1802.03932, doi:doi.org/10.1145/3208976.3208998.
- [LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *J. ACM*, 39:859–868, October 1992. doi:10.1145/146585.146605.
- [LRSW99] Anna Lysyanskaya, Ron Rivest, Amit Sahai, and Stefan Wolf. Pseudonym systems. In Howard M. Heys and Carlisle M. Adams, editors, *SAC 1999: 6th Annual International Workshop on Selected Areas in Cryptography*, volume 1758, 1999. doi:10.1007/3-540-46513-8_14.

- [Mat08] Todd Mateer. *Fast Fourier transform algorithms with applications*. PhD thesis, Clemson University, 2008. URL: <https://cr.yp.to/f2mult/mateer-thesis.pdf>.
- [Nus80] Henri J. Nussbaumer. Fast polynomial transform algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28:205–215, 1980. URL: <https://api.semanticscholar.org/CorpusID:121556373>, doi:10.1109/TASSP.1980.1163372.
- [Pai24] Pairing crypto bbs implementation. https://github.com/mattrglobal/pairing_crypto, 2024.
- [Per23] Personae. <https://github.com/personaelabs/spartan-ecdsa.git>, 2023.
- [RWGM23] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. zk-creds: Flexible anonymous credentials from zksnarks and existing identity infrastructure. In *2023 IEEE Symposium on Security and Privacy*, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press. doi:10.1109/SP46215.2023.10179430.
- [SJB87] H. Sorensen, D. Jones, and C. Burrus. Real-valued algorithms for the fft. In *ICASSP '87. IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 12, pages 1831–1834, 1987. doi:10.1109/ICASSP.1987.1169490.
- [Skl60] Jack Sklansky. Conditional-sum addition logic. *IRE Trans. Electron. Comput.*, 9:226–231, 1960. URL: <https://api.semanticscholar.org/CorpusID:41264042>, doi:10.1109/TEC.1960.5219822.
- [Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part II*, volume 8043, 2013. doi:10.1007/978-3-642-40084-1_5.
- [Tha22] Justin Thaler. Proofs, arguments, and zero-knowledge. Manuscript, 2022.
- [vdH04] Joris van der Hoeven. The truncated fourier transform and applications. In *Proceedings of the 2004 International Symposium on Symbolic and Algebraic Computation*, ISSAC '04, page 290–296, New York, NY, USA, 2004. Association for Computing Machinery. doi:10.1145/1005285.1005327.
- [vdH05] Joris van der Hoeven. Notes on the truncated Fourier transform, 2005. URL: <https://www.texmacs.org/joris/tft/tft.pdf>.
- [vzGG96] Joachim von zur Gathen and Jürgen Gerhard. Arithmetic and factorization of polynomial over $\mathbb{F}_2$ (extended abstract). In *Proceedings of the 1996 International Symposium on Symbolic and Algebraic Computation*, ISSAC '96, pages 1–9, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/236869.236882.
- [WHV24] Ruihan Wang, Carmit Hazay, and Muthuramakrishnan Venkatasubramanian. Ligetron: Lightweight scalable end-to-end zero-knowledge proofs post-quantum zk-snarks on a browser. In *2024 IEEE Symposium on Security and Privacy*, San Francisco, CA, USA, May 19–23, 2024. IEEE Computer Society Press. doi:10.1109/SP54263.2024.00086.

- [WJB⁺17] Riad S Wahby, Ye Ji, Andrew J Blumberg, abhi shelat, Justin Thaler, Michael Walfish, and Thomas Wies. Full accounting for verifiable outsourcing. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, 2017. doi:[10.1145/3133956.3133984](https://doi.org/10.1145/3133956.3133984).
- [WOS⁺25] Anna Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient proofs of possession for legacy signatures. In *IEEE S&P 2025*, 2025. doi:[10.1109/SP61157.2025.00080](https://doi.org/10.1109/SP61157.2025.00080).
- [WTas⁺18] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *eprint/2017/1132 and 2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 2018. doi:[10.1109/SP.2018.00060](https://doi.org/10.1109/SP.2018.00060).
- [XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510, 2022. doi:[10.1007/978-3-031-15985-5_11](https://doi.org/10.1007/978-3-031-15985-5_11).
- [ZXZS19] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. Transparent polynomial delegation and its applications to zero knowledge proof. In *2020 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 2019. doi:[10.1109/SP40000.2020.00052](https://doi.org/10.1109/SP40000.2020.00052).

A The MDOC standard

MDOCs are specified in ISO 18013-5. Here we summarize the relevant portion that concerns a binary string which can be parsed into a hierarchical MSO object of the form:

```
{
  "version": "1.0", "digestAlgorithm": "SHA-256",
  "docType": "org.iso.18013.5.1.mDL",
  "valueDigests": {
    "org.iso.18013.5.1": {
      {13: h'B628...69D2',
       14: h'8CFE...604C',
       ...
    },
    "org.iso.18013.5.1.aamva": {
      15: h'1034...402A',
      ...
      8: h'7AC6...8485'}
  },
  "deviceKeyInfo": {
    "deviceKey": {1: 2, -1: 1,
                  -2: h'7B8F...F321',
                  -3: h'859E...772C'}
  },
  "validityInfo": {
    "signed": 0("2023-10-11T13:18:15Z"),
    "validFrom": 0("2023-10-11T13:18:15Z"),
    "validUntil": 0("2023-11-10T13:18:15Z")
  }
}
```

In our context, an MDOC credential consists of a binary serialization of this MSO object that is signed by an IdentityIssuer (II), whose public signing key PK_{II} is well-known; denote the signature (r, s) . The notation $MSO[valueDigests][org.iso.18013.5.1][14]$ represents the SHA-256 hash of a tuple consisting of the strings abbreviated as $(id, nonce, attrid, attrvalue)$:

```
"digestID": 14,
"random": h'8c082af3d5d78b98bcc00bd26fae5130',
"elementIdentifier": "age_over_18",
"elementValue": true
```

A user can disclose the `age_over_18` attribute in the MDOC credential by providing a signature of the MSO and the SHA256 preimage (consisting of an encoding of those 4 strings) to the 14th hash included in correct namespace list of the `valueDigests` field of the MSO. This asserts that the property “`age_over_18`” was included in the bundle that the IdentityIssuer signed, while hiding the other attributes that were signed.

To prevent users from breaking soundness by copying credentials, the MSO contains a device-bound public key PK_d in fields $MSO[deviceKeyInfo][deviceKey][-2]$ and $[-3]$. The secret key for this public key is stored in a *secure element* on the user’s device. When the MDOC was issued, the IdentityIssuer verifies that the SK_d is attested by a *secure element*. When a user presents this MDOC, in addition to (r, s) and the preimage of the attribute, the user’s device computes a fresh signature on a transcript of the identity request under the PK_d signing key. This requirement for a signature to be computed by *trusted hardware* makes it impractical to use BBS+ because secure elements do not support pairing-friendly curves.